

**Die Grenzen von cachebasierten Covert-Channels im Kontext von Transient-Execution:
ein technischer Überblick**

Bachelor-Thesis
an der
Leibniz-Fachhochschule
im Rahmen des Studiengangs
IT-Security
zum Bachelor of Science IT-Security (B.Sc.)

vorgelegt von
Der Autor
aus
Adresse des Autors
der.autor@example.com

Matr.-Nr.: 5

Erstgutachter: Gutachter
Zweitgutachter: Zweitgutachter

Abgabe: 1.6.2026

Abkürzungsverzeichnis

CPU Central Processing Unit

ALU Arithmetic Logic Unit

RAM Random Access Memory

SRAM Static Random Access Memory

DRAM Dynamic Random Access Memory

ISA Instruction Set Architecture

SAC Set Associative Cache

LLC Last Level Cache

CbCC Cache-basierte Covert-Channel

PC Program Counter

RAW Read After Write

SMT Simultaneous Multi-Threading

ROB Reorder Buffer

μ -op Mikroinstruktion

LFB Line Fill Buffer

OoO Out-of-Order

POC Proof of Concept

TEA Transient Execution Angriff

VM Virtuelle Maschine

Inhaltsverzeichnis

Abkürzungsverzeichnis	I
Inhaltsverzeichnis	II
Abbildungsverzeichnis	IV
1 Einführung	1
1.1 Schutzziele und Informationssicherheit	1
1.2 Side-Channel und Transient Execution Angriffe (TEAs)	1
1.3 Zielstellung	2
2 Theoretischer Hintergrund	3
2.1 Central Processing Unit (CPU) und Speicher Architektur	3
2.1.1 Computer Instruktionen und Ressourcen	3
2.1.2 Speicher und Cache Architektur	4
2.1.3 Pipelined Prozessoren	6
2.1.4 Transient Execution	7
2.2 Cache-basierte Covert-Channel (CbCC)	8
2.2.1 Cache als Informationsmedium	9
2.2.2 Realisierung verschiedener Cache-Covert-Channel	11
2.3 Transient Execution Angriffe (TEAs)	12
2.3.1 Einordnung verschiedener Angriffe	13
2.3.2 Zeitlicher Aufwand von TEAs	14
2.4 Mitigationen von Spectre und CbCC	14
3 Methodik und Auswertung	16
3.1 Threat-Model	16
3.2 Methodik der experimentellen Betrachtung	16
3.3 Auswertung	19
3.4 Theoretische Laufzeitbetrachtung	21
3.5 Diskussion	23
4 Fazit	26
4.1 Ergebnisse	26
4.2 Ausblick	26
Literatur	28
Anhang	32
A Mikroinstruktionen und Speicherzugriffe	32
B Grundlegende Cache Architekturen	32
C Mehrschichtiger Cache	33
D Spekulative Ausführung von Branch Instruktionen	33

E	Weitere technische Möglichkeiten für Covert-Channel in der Prozessorarchitektur	34
F	Erläuterung zu verschiedenen CbCC Techniken	35
G	Erläuterung zu der Messung, ob eine Adresse gecacht ist	36
H	Abbildungen zur theoretischen Laufzeit einer Übertragung bei TEAs	37
I	Programmausschnitte des Test-Programms	37
J	Cache Spezifikationen der verwendeten CPUs	38
K	Messungen des Test-Programms	41

Abbildungsverzeichnis

1	Cache-Architekturen der getesteten CPUs (nach Anhang J). Die Caches sind jeweils nur für zwei Kerne dargestellt, um das Konzept der Architektur zu veranschaulichen.	6
2	Phases of transient execution attacks. (Xiong und Szefer, 2022, S. 2)	13
3	Aufbau des Test-Programms	17
4	$D(n_a)$ mit $N_T = 50$, $T_T = 20$, $L_E = 100$, $T_P = 50$, $T_R = 120$ und $n_{bmax} = 8$.	37
5	Die erste Ableitung der Durchsatz-Funktion $D(n_a)$	37
6	Messergebnisse des Test-Programms auf verschiedenen CPUs (Encodierung bitweise)	42
7	Messergebnisse des Test-Programms auf verschiedenen CPUs (Encodierung Array-Index)	43
8	Messergebnisse zur Laufzeit des Test-Programms auf verschiedenen CPUs (Encodierung bitweise)	44
9	Messergebnisse zur Laufzeit des Test-Programms auf verschiedenen CPUs (Encodierung Array-Index)	44

1 Einführung

1.1 Schutzziele und Informationssicherheit

Die IT-Sicherheit befasst sich mit dem Schutz von Informationen sowie der Systeme, die diese Informationen verarbeiten. Es wird ein Zustand angestrebt, bei dem das betrachtete System die definierten Sicherheitsziele erfüllt (Von Faber, 2021, S. 16). Als die drei wichtigsten Schutzziele wird meist die CIA-Triade herangezogen (Liedtke, 2022, S. 21). Das Akronym CIA steht für „Confidentiality“ (Vertraulichkeit), „Integrity“ (Integrität) und „Availability“ (Verfügbarkeit) (Von Faber, 2021, S. 12).

Eine zentrale Herausforderung besteht darin, dass auf jeder Ebene des Systems Schwachstellen existieren können, die zur Verletzung dieser Schutzziele führen können. (Liedtke, 2022, S. 19)

Zur Gewährleistung von Informationssicherheit, werden sowohl organisatorische als auch technische Aspekte betrachtet. Der Fokus liegt dabei häufig auf Software, sowie auf der Applikations- und Betriebssystemebene. Schwachstellen in darunter liegenden Bereichen, beispielsweise bei physischen Zugriffen, können die Sicherheit des gesamten Systems schwächen.

Um Hardware Ressourcen Effizient zu nutzen, werden diese zwischen verschiedenen Aufgaben geteilt. (Kamp und Watson, 2004, S. 44) Entsprechend existieren verschiedenste Mechanismen, um den Zugriff eines Programms auf die Ressourcen anderer Programme zu verhindern und so die Schutzziele zu gewährleisten. Diese Isolation kann in Software beispielsweise als Virtuelle Maschine (VM) oder durch das Betriebssystem realisiert werden, geschieht aber häufig in enger Verknüpfung mit der Hardware.

1.2 Side-Channel und Transient Execution Angriffe (TEAs)

Mit der Entdeckung praktischer Cache-basierter Side-Channels, wie beispielsweise in Bernstein, 2005, wurde ein neuer Angriffsvektor sichtbar, der auf der konzeptionellen Funktionsweise moderner Computersysteme beruht. Das Problem mit dieser Art von Angriff ist, dass Schwachstellen in den Bereichen zwischen der Hard- und Software ausgenutzt werden, die in verschiedensten CPU Architekturen und Implementierungen auftreten. Mit der Entdeckung von TEAs durch Kocher et al., 2019 und Lipp et al., 2020 wurde dieser Angriffsvektor weiter vergrößert. Da diese Angriffe grundlegende Konzepte moderner CPUs ausnutzen, betrifft diese Art an Schwachstellen sowohl x86 Intel und AMD als auch ARM CPUs-Architekturen. (Canella et al., 2019, S. 253, 255) TEAs nutzen dieselben Eigenschaften wie Cache-basierte Side-Channel für sogenannte Covert-Channel, um die Informationen wiederzuerlangen. Obwohl außer CbCC potenziell auch contenti-onbasierte Covert-Channel (Z. Wang und Lee, 2006, S. 2) eingesetzt werden könnten, wurden bei Proofs of Concepts (POCs) von TEAs bisher grundsätzlich CbCCs verwendet. (Kocher et al., 2019, S. 4, 6, Lipp et al., 2020, S. 4, Schwarz et al., 2019, S. 7, Kim, Genkin und Yarom, 2025, S. 6, Van Schaik et al., 2019, S. 91).

Da TEAs eine hohe Komplexität und starke Hardware Abhängigkeit aufweisen, wird ihre praktische Umsetzbarkeit außerhalb kontrollierter Laborumgebungen häufig hinter-

fragt. Aus demselben Grund wird das von diesen Angriffen ausgehende Risiko hinterfragt. (Shepherd et al., 2022, S. 293, 296) Besonders bei browserbasierten Angriffen, wie beispielsweise Kim et al., 2023, Kim, Genkin und Yarom, 2025 und Kim, Chuang et al., 2025, die eine relativ hohe Laufzeit aufweisen und einen passiven Nutzer voraussetzen, besteht die Frage, ob diese Angriffe praktisch von einem Angreifer genutzt werden könnten. Dabei könnte derselbe Angriff ein deutlich höheres Risiko darstellen, insofern seine Laufzeit erheblich reduziert werden kann.

Da das grundlegende Problem ohne deutliche Performanceeinbußen nicht verhindert werden kann, existieren verschiedene Ansätze zur Entwicklung von Mitigationen gegen Covert-Channels während Transient Execution. Dazu zählen sowohl automatische statische Analysen von Programmen, mit denen sich mögliche Gadgets für solche Angriffe identifizieren lassen (Brotzman et al., 2021), als auch Mitigationen gegen spezifische Covert-Channel während Transient Execution (Cheng et al., 2024). Eine generelle Mitigation gegen Transient Execution und Cache-basierte Side-Channel Angriffe ist jedoch praktisch nicht möglich, ohne die ausgenutzten Optimierungen vollständig zu entfernen. Die einzige allgemeine Möglichkeit besteht darin Störungen durch Änderungen in den Cache Zuständen auszulösen, um eine Übertragung deutlich zu erschweren oder unmöglich zu machen. (Z. Wang und Lee, 2006, S. 6, Su und Zeng, 2021, S. 11) Da TEAs allerdings nicht ausschließlich auf CbCC angewiesen sind und Störungen allein nicht den Angriff verhindern, werden häufig weitere Maßnahmen betrachtet, die allerdings nur spezifische Angriffsszenarien abdecken. Aus diesem Grund bleiben solche Angriffe ein Risiko, welches nicht vollständig mitigiert werden kann. Daraus ergibt sich die Frage, welche Angriffe unter realistischen Bedingungen tatsächlich praktikabel sind.

1.3 Zielstellung

Ziel dieser Arbeit ist es, einen Überblick über die technischen Möglichkeiten von CbCC hinsichtlich ihrer Grenzen im Kontext von Transient Execution zu erarbeiten.

Hierzu werden zwei Ansätze zur Implementierung von CbCC sowie verschiedene bekannte Techniken in Kombination mit Transient Execution grundlegend verglichen. Dabei sollen Parameter identifiziert werden, die für die Übertragungsgeschwindigkeit eines CbCC entscheidend sind. Auf dieser Grundlage soll eine Einschätzung der Grenzen solcher CbCC auf gegebener Hardware ermöglicht werden.

2 Theoretischer Hintergrund

2.1 CPU und Speicher Architektur

Computerarchitekturen bestehen aus mehreren Ebenen zur Abstraktion der Hardware, von einzelnen Transistoren bis hin zu einem Betriebssystem und Benutzerprogrammen. Die Funktionsweise von CbCC und Transient Execution befindet sich auf der Ebene der CPU-Architektur, ihrer Mikroarchitektur sowie der darunter liegenden Hardware. Eine Architektur ist die strukturelle Sicht des Computers, die dem Softwareentwickler präsentiert wird. Anstatt diese Architektur direkt in Hardware zu bauen und Entwicklern zur Verfügung zu stellen, wird bei modernen Prozessoren eine zusätzliche Abstraktionsschicht in Form von Mikrocode eingefügt. Diese ermöglicht die Implementierung von Instruktionen in Software und damit auch das nachträgliche patchen von Bugs. (SARANGI, 2021, S. 14, 379) Dementsprechend sind für jede Schicht nur bedingt die Vorgänge der darunter liegenden Schicht sichtbar. Sowohl CbCC als auch TEAs nutzen Vorgänge in ihnen nicht zugänglichen, unteren Schichten aus, um in der Regel Informationen aus anderen Software Bereichen zu lesen. Aufgrund der Hardwarenähe der Schichten, die ausgenutzt werden ist das Erkennen entsprechender Informationsflüsse sehr schwer.

2.1.1 Computer Instruktionen und Ressourcen

Eine grundsätzliche Rechnerarchitektur ist heute als die „Von-Neumann-Architektur“ bekannt. Die zentrale Komponente dieser Architektur ist die CPU, welche für reine arithmetische Berechnungen (Arithmetic Logic Unit (ALU)) und für die Programminterpretation zuständig ist (Control Unit). Des Weiteren enthält diese Architektur einem Speicher (engl. Memory) sowie Geräte für Eingabe und Ausgabe. Dieser Speicher wird auch Arbeitsspeicher beziehungsweise Random Access Memory (RAM) genannt. Bei modernen Computern ist der RAM, aufgrund seiner Größe im Verhältnis zur Geschwindigkeit der Rechenoperationen vergleichsweise langsam. Aus diesem Grund verwendet die CPU selbst interne Register mit einer Kopie der aktuell benötigten Werte aus dem RAM. (SARANGI, 2021, S. 35)

Ein einzelner Computer kann mehrere Prozessoren besitzen, die Programme parallel ausführen können. Wenn sich mehrere Prozessoren auf demselben Chip befinden, werden diese Kerne genannt. Mittlerweile gehört der größte Teil der Computersysteme zu den Multicore-Systemen mit eigenen Speicherzugriffen. (SARANGI, 2021, S. 569, 578)

Die unterste Softwareebene, die für den Entwickler bereitgestellt wird, ist die Instruction Set Architecture (ISA) einer CPU. Diese beschreibt ursprünglich die Befehle, die von der CPU in Hardware ausgeführt werden können. Während arithmetische Operationen dieser Instruktionen direkt auf den Registern ausgeführt werden, wird die Speicherarchitektur auf primär die Instruktionen „load“ und „store“ abstrahiert. Zwar existieren weitere speicherrelevante Instruktionen, diese hängen jedoch von der jeweiligen ISA und teilweise auch von der konkreten Hardwareimplementierung ab. Bei modernen Prozessoren ist die ISA außerdem nicht direkt in der Hardware implementiert, sondern wird durch sogenannte Mikroinstruktionen (μ -ops) abstrahiert. Da diese für die vorliegende Arbeit

nur von untergeordneter Bedeutung sind, werden sie in Anhang A näher beschrieben. Entsprechend arbeitet die ISA mit dem architektonischen, während die Mikroinstruktionen mit dem mikroarchitektonischen Zustand der CPU arbeitet. (SARANGI, 2021, S. 19, 373, 379) Auch wenn verschiedene ISAs existieren und die besprochenen Konzepte auf die meisten Implementierungen von CPUs zutreffen, wird in dieser Arbeit – sofern nicht anders angegeben – primär die x86-64 ISA betrachtet. Im Folgenden ist außerdem mit Speicher stets der nicht persistente RAM des Computers gemeint, der von der CPU während der Ausführung verwendet wird.

Da sich bei der Von-Neumann-Architektur Instruktionen und Daten im selben Speicher befinden, wird der Speicher überwiegend als ein großes Array mit aufsteigenden Adressen betrachtet. Adressiert wird dabei jeweils ein Byte. Eine Folge von Instruktionen, die von einem Computer ausgeführt werden soll, wird als Programm bezeichnet und befindet sich genauso wie die Daten im Speicher. Um ein Programm auszuführen, müssen die Instruktionen zunächst von der CPU geladen (engl. fetch), anschließend von der Hardware interpretiert und schließlich ausgeführt werden. Die Adresse der aktuell auszuführenden Instruktion wird als Programmzähler (engl. Program Counter (PC)) bezeichnet. (Mandl, 2020, S. 74, 188)

2.1.2 Speicher und Cache Architektur

Ein zentrales Problem moderner Computer ist die Abhängigkeit von Zugriffen auf den RAM. Die Speichergroße der CPU-internen Register ist begrenzt und reicht für komplexere Programmabläufe nicht aus. Deshalb befinden sich sowohl Programme als auch deren Werte im Speicher und müssen durch „load“ und „store“ Instruktionen mit den Registern synchronisiert werden. Dementsprechend besteht ein erheblicher Anteil jedes Programms aus Speicher Zugriffen. Allerdings ist die Menge an Speicher auf demselben Chip wie der CPU aus Platzgründen deutlich begrenzt.

Grundsätzlich lassen sich zwei für moderne Rechnersysteme relevante RAM-Technologien unterscheiden: Static Random Access Memory (SRAM) und Dynamic Random Access Memory (DRAM). An sich ist SRAM die deutlich überlegene Technologie mit signifikant geringeren Zugriffszeiten. Jedoch ist SRAM besonders bei der Anordnung in größeren Mengen erheblich größer und teurer. Auch wenn die modernsten Server Prozessoren heute bis zu einem Gigabyte SRAM („Server Processor Specifications“, o. D.) auf dem Chip erreichen, ist dies besonders im Verhältnis zu der Anzahl der Kerne des Prozessors sehr gering. Der Speicher auf dem CPU-Chip ist allerdings nur ein Cache, der einen Teil des gesamten Speichers des Computers hält und die Zugriffszeit für möglichst viele Zugriffe minimieren soll. Dieser Cache ist nötig, da seit dem Ende des 20. Jahrhunderts die Speicherzugriffszeit die Rechenzeit der CPU deutlich übersteigt. (Handy, 1993, S. 37, 38, Hat, 2007, S. 13)

Der restliche Programmspeicher befindet sich außerhalb der CPU und muss über einen Bus angesteuert werden. Während ein idealer Speicher eine Zugriffszeit von einem CPU-Zyklus hätte und damit praktisch instantan wäre, ist die reale Zugriffszeit abhängig von der Speicherebene, die angefragt wird. Diese Zeit liegt in der Regel zwischen ein bis zwei und wenigen 100 Zyklen (Yarom und Falkner, 2014, S. 723, 724), je nachdem

welche Speicher Schicht angefragt wird. (Hat, 2007, S. 16, SARANGI, 2021, S. 534)

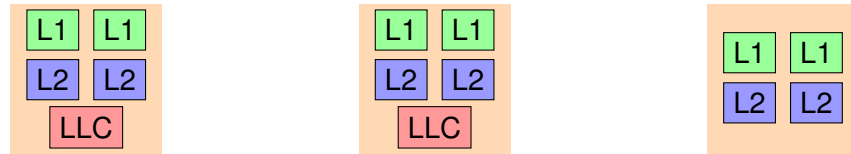
Da der Cache nur ein Abbild eines Teils des gesamten Speichers ist, muss dieser so implementiert werden, dass sich bei Bedarf beliebige Adressen in den Cache schreiben und von dort lesen lassen. Gleichzeitig muss ein effizienter Mechanismus implementiert werden, um auf physischer Ebene anhand der Speicheradresse den Wert der Adresse zu ermitteln. Ist eine Adresse bereits im Cache vorhanden und kann direkt zurückgegeben werden wird dies ein Cache „Hit“, andernfalls ein „Miss“ genannt (SARANGI, 2021, S. 514). Das Entfernen einer Adresse beziehungsweise eines Blocks wird „Eviction“ beziehungsweise als Verb „evict“ genannt.

Empirisch zeigt sich, dass dicht nebeneinander liegende Adressen häufig im Zusammenhang verwendet werden. Diese Beobachtung wird als räumliche Lokalität bezeichnet. Aus diesem Grund speichern Caches Daten nicht bytewise, sondern in Blöcken. Anstatt des Ladens einer einzelnen Adresse wird also ein fester Speicherbereich – typischerweise mit einer Größe zwischen 32 und 128 Byte – in den Cache geladen. Der Cache ermittelt anhand der zu ladenden Adresse den gesuchten Block und gibt den darin gesuchten Wert zurück. Um von der Speicheradresse auf den physischen Cache Eintrag schließen zu können, wird eine Mappingmethode verwendet (vgl. B). In der Praxis wird häufig ein Set Associative Cache (SAC) verwendet. Dabei ist jede Speicheradresse einer begrenzten Menge von physischen Cache-Einträgen, einem sogenannten „Set“, zugeordnet. Die Anzahl an physischen Einträgen je Set wird häufig, wie beispielsweise in Anhang J, als „Ways“ bezeichnet. Die Einträge eines Sets werden über eine Setadresse angesteuert, die von der physischen Adresse abgeleitet wird. Dies kann durch das Assoziieren eines Teils der Speicheradresse mit der Setadresse geschehen. Zum Beispiel könnten die ersten N Bit einer Adresse die Setadresse darstellen. Dabei wird das Mapping meist so gewählt, dass Adressen, die sich im selben Set befinden, nicht direkt nebeneinander liegen. Denn Adressen im selben Set konkurrieren gewissermaßen um den Cache und nach dem Prinzip der räumlichen Lokalität werden Adressen, die dicht beieinander liegen, häufig zur gleichen Zeit genutzt. (SARANGI, 2021, S. 515, 516, Handy, 1993, S. 16)

Eine Konsequenz aus dieser Architektur ist die Existenz von sogenannten Eviction-Sets. Ein Eviction-Set ist eine Menge an physischen Adressen, die alle dem gleichen Cache-Set zugeordnet sind. Da diese Adressen bewusst nicht nah beieinander gewählt werden, ist es zudem nicht unwahrscheinlich, dass verschiedene Programme virtuelle Adressen besitzen, deren physische Adressen demselben Eviction-Set angehören. (Vila et al., 2019, S. 40, 41)

Aufgrund der Größe der SRAM Zellen des Cache nimmt mit zunehmender Cache Größe auch die Zugriffszeit zu. Aus diesem Grund ist der Cache in Prozessoren in mehrere Ebenen mit unterschiedlichen Größen geteilt. Die meisten Prozessoren haben 3 verschiedene Cache „Layer“, die mit L1, L2, L3 bezeichnet werden. Der letzte Cache vor dem RAM wird Last Level Cache (LLC) genannt. (Slomka und Glaß, 2023, S. 388, SARANGI, 2021, S. 596)

Um die logische Implementierung dieser Cache Layer zu vereinfachen, kann ein Cache „inclusive“ gegenüber einer höheren Schicht sein. Das heißt, dass alle Blöcke, die dieser Cache enthält, sich auch in dem darüberliegenden Cache befindet. Inclusive



(a) Intel® Core™ i7-6700K (b) Intel® Core™ i7-7700HQ (c) AMD A10-7850K APU

Abbildung 1: Cache-Architekturen der getesteten CPUs (nach Anhang J). Die Caches sind jeweils nur für zwei Kerne dargestellt, um das Konzept der Architektur zu veranschaulichen.

CPU-Speichersysteme sind aufgrund der einfacheren Implementierung deutlich weiter verbreitet. Einige neuere Prozessoren nutzen mittlerweile allerdings non-inclusive Cache Layer (SARANGI, 2021, S. 513, Rauscher et al., 2025, S. 4).

Darüber hinaus können Caches als „write-through“ oder „write-back“ Caches implementiert werden. Write-through beschreibt das Korrigieren eines geschriebenen Wertes im Speicher, sobald dieser im Cache geändert wird. Bei write-back dagegen, wird erst bei der Eviction des Blocks der Wert in den restlichen Speicherschichten neu gesetzt. (SARANGI, 2021, S. 526, 527) Bei mehreren Kernen kann ein Cache „private“, das bedeutet nur für einen Teil der Kerne zugreifbar, oder „shared“, das heißt für alle Kerne zugreifbar, sein. Wenn ein Cache-Layer private ist, bedeutet das außerdem, dass für jeden Kern dieser Cache separat physisch implementiert ist. (SARANGI, 2021, S. 593, 594) Auch wenn die im folgenden betrachteten CPUs Instruktionen und Daten in demselben Speicher halten, ist es üblich, den L1 Cache in einen Cache für Instruktionen (L1i) und einen für Daten (L1d) zu teilen. (Schwarz et al., 2019, S. 3) Diese Unterteilung ist für CbCC weniger relevant, weswegen im folgenden alle Caches so betrachtet werden, als würden sie sowohl Daten als auch Instruktionen speichern. Je nachdem, welche Architektur im Cache-Design eines Prozessors implementiert ist, sind verschiedene CbCCs nutzbar. (Rauscher et al., 2025, S. 4, 5) Eine in der Vergangenheit häufige Cache-Architektur, beinhalteten zwei bis vier Layer mit ein bis drei private Layern und einem shared LLC. In Abbildung 1 sind die Cachearchitekturen der getesteten CPUs dargestellt; darunter befindet sich auch diese Architektur. Die verschiedenen Cache-Implementierungen sind in Anhang C detaillierter erläutert.

2.1.3 Pipelined Prozessoren

Um Instruktionen auszuführen werden diese in μ -ops übersetzt, die sequentiell ausgeführt werden müssten. Alle μ -op durchlaufen dabei verschiedene Phasen. Beispielsweise müssen die μ -ops und ihre Operanden aus dem Speicher in die Register geladen, in der ALU ausgeführt und das Ergebnis anschließend in ein Zielregister geschrieben werden. Das Ergebnis dieser Architektur sind einzelne Einheiten, die miteinander Verbunden sind und in einer festen Reihenfolge arbeiten. In jedem Taktzyklus der CPU wird eine Phase einer μ -op abgearbeitet. Da die μ -ops allerdings sequenziell die Phasen durchlaufen bleibt eine Menge Zeit in einzelnen Einheiten ungenutzt. Aus diesem Grund werden Prozessoren als Pipeline gebaut, in der mehrere, aufeinanderfolgende μ -ops, parallel in verschiedenen Phasen ausgeführt werden. (SARANGI, 2021, S. 361, 362)

Die Idee des Pipelining besteht darin, die Einheiten der CPU in sequenzieller Form anzuordnen, sodass in jedem Zyklus jede Phase eine μ -op verarbeitet. Jeder Übergang zwischen zwei Einheiten wird dabei durch Register realisiert, die die maximal nötigen Daten für die folgenden Phasen zwischenspeichern. Die Phasen werden bei modernen CPUs immer granularer unterteilt, um mehr μ -ops gleichzeitig in kürzer werdenden Zyklen verarbeiten zu können. (SARANGI, 2021, S. 414–416)

Ein Problem dieser Struktur besteht darin, dass μ -ops Abhängigkeiten zu früheren μ -ops haben können und die Verarbeitung der vorherigen μ -op zumindest teilweise abgeschlossen sein muss. Ein typisches Beispiel dafür sind Branch-Instruktionen. Diese haben die Aufgabe den Control Flow, also welche Programmteile ausgeführt werden, dynamisch zu ändern. Dies geschieht, indem der PC auf eine neue Zieladresse gesetzt wird. Der neue Wert des PC verweist entweder auf einen anderen Programmteil als den aktuell ausgeführten, dann wird der Branch „genommen“, oder der Programmablauf wird nicht geändert und der PC wird auf die nächste Instruktion gesetzt, in diesem Fall wird der Branch nicht „genommen“. (Kaeli und Yew, 2005, S. 31, 34)

Die Veränderung des PC erfolgt an einem Punkt in der Pipeline, wo die nächsten μ -ops bereits in die Pipeline geladen und verarbeitet werden sollten. Hinzu kommt, dass ein Prefetching-Mechanismus Instruktionen aus dem Speicher lädt, bevor diese ausgeführt werden. Dies ist notwendig, da das Laden der Instruktionen zum Zeitpunkt ihrer Ausführung von der Speicherlatenz abhängen würde und die Pipeline erhebliche Leerlaufzeiten hätte. (Kaeli und Yew, 2005, S. 9–11) Diese Abhängigkeit wird als Control-Hazard bezeichnet und muss in dem Aufbau der CPU berücksichtigt werden.

Ein sogenanntes Data-Hazard kann dagegen auftreten, wenn eine spätere Instruktion Daten einer früheren Instruktion verwendet, obwohl diese frühere Instruktion den Schritt des Schreibens der Daten in ein Register noch nicht erreicht hat. Diese Art von Abhängigkeit wird auch als Read After Write (RAW)-Abhängigkeit bezeichnet. (SARANGI, 2021, S. 423–427)

Bei einer Branch Instruktion müsste also der Prefetching-Mechanismus und die gesamte Pipeline warten, bis die Branch-Entscheidung getroffen und der PC angepasst wurde. Je länger die Pipeline ist und je später die Branch-Entscheidung erfolgt, desto länger ist diese Wartezeit. (Kaeli und Yew, 2005, S. 29–36)

2.1.4 Transient Execution

Die Pipeline-Struktur führt zum Ausführen von μ -ops, ohne dass vorherige μ -ops beendet oder alle Abhängigkeiten aufgelöst sind. Die CPU speichert die Zustände der einzelnen μ -ops und übernimmt (engl. committed) die architektonischen Änderungen in der korrekten Reihenfolge, sobald alle Abhängigkeiten aufgelöst wurden. Führt eine Abhängigkeit zu einem Fehler, werden die Änderungen verworfen (engl. squashed) und die μ -ops wurden nur „flüchtig“, also transient, ausgeführt. Änderungen in dem Bereich der Mikroarchitektur bleiben allerdings teilweise erhalten. (Canella et al., 2019, S. 251) Für das Speichern des Zustands der μ -ops und das Committed nach dem Auflösen der Abhängigkeiten ist der sogenannte Reorder Buffer (ROB) zuständig. (SARANGI, 2021, S. 493, 494) Da dieser Speicher begrenzt ist kann nur eine gewisse Menge an μ -ops transient

ausgeführt werden (Mosier et al., 2025, S. 6).

Ein weiterer Optimierungsmechanismus, der eng mit Pipelining zusammenhängt, ist Out-of-Order (OoO)-Execution. Ziel ist es, Instruktionen ohne Data Hazards effizienter auszuführen. Dazu werden voneinander unabhängige Instruktionen in veränderter Reihenfolge ausgeführt, um die Auslastung der CPU Einheiten zu maximieren. Durch das Ausführen späterer Instruktionen vor den zuvor kommenden Instruktionen führt allerdings auch dieses Konzept zu Transient Execution. Wenn beispielsweise eine im Programm frühere Instruktion zeitlich nach folgenden Instruktionen ausgeführt wird und einen Fehler auslöst, können diese späteren Instruktionen bereits ausgeführt worden sein, obwohl sie im eigentlichen Programmablauf erst nach dem Fehler folgen würden. Die im Programm nachfolgenden Instruktionen wurden zum Zeitpunkt des Fehlers bereits ausgeführt und trotz des zurücksetzen des Zustands können Informationen durch Covert- beziehungsweise Side-Channel übertragen werden. (Lipp et al., 2020, S. 5, Cheng et al., 2024, S. 633)

Spekulative Ausführung ist eine spezielle Form der transient Execution, bei der die CPU Control- oder Data-Hazards durch das Vorhersagen von Abhängigkeiten auflöst. Die Anzahl der Instruktionen beziehungsweise die Zeitspanne, über die die CPU Instruktionen ausführt, die später verworfen werden, wird als die Größe oder Tiefe des spekulativen beziehungsweise transient Fensters bezeichnet. Diese Zeit hängt von der Größe des ROB sowie der Dauer ab, bis die zugrunde liegenden Abhängigkeiten aufgelöst werden können. Die Größe dieses Fensters ist beispielsweise besonders groß, wenn eine benötigte Adresse nicht im Cache vorhanden ist und aus dem RAM geladen werden muss. In diesem Falle ist das Fenster nur durch die Größe des ROB begrenzt. (Mosier et al., 2025, S. 6) Transient Execution wird am Beispiel von spekulativer Branch-Ausführung in Anhang D näher erläutert. (Kaeli und Yew, 2005, S. 136)

2.2 Cache-basierte Covert-Channel (CbCC)

Soll ein Programm von einem Computer ausgeführt werden, ohne dass ihm vollständig vertraut werden kann, erhält es zumindest Eingabeparameter und ohne weitere Restriktionen hardware-spezifische und ähnliche Informationen. Aus Sicht der Sicherheit besteht die Möglichkeit, die Zugriffe des Programms auf absolut nötige Ressourcen zu beschränken. Allerdings auch dann könnte das Programm Informationen erhalten, die es zwar verarbeiten, aber nicht an Dritte weiterleiten soll. Das Verhindern solcher Informationsflüsse wurde bereits von Lampson, 1973 als das „Confinement Problem“ beschrieben. Es bezieht sich auf das Übertragen von Informationen eines Programms nicht an das Programm oder den Nutzer, der es aufgerufen hat, sondern an den „Owner“, also beispielsweise den Ersteller des Programms. Die verschiedenen Wege der Informationsübertragung lassen sich dabei in drei verschiedene Kategorien einordnen: in Speicher, legitime Kanäle und sogenannte verdeckte, also „Covert“-Channel, die implizit Informationen über nicht dafür vorgesehene Zustände wie beispielsweise die Prozessorauslastung übertragen. (Lampson, 1973, S. 613, 614)

Wird auf diesem Wege eine Information nicht absichtlich von einem „Insider“ an einen „Outsider“ gesendet, sondern kann dieser Outsider aus der Beobachtung dieser Kanäle

auf Informationen des inneren Zustands eines Systems schließen, dann handelt es sich um einen Seitenkanal, also einen Side-Channel. (Maurice et al., 2015, S. 1, Z. Wang und Lee, 2006, S. 1)

Covert-Channel können grob in zwei Kategorien eingeteilt werden, deren mögliche Verwendung stark vom jeweiligen Kontext abhängt. Paccagnella et al., 2021, S. 3 unterscheiden nach eviction- und contention(Konkurrenz)-basierten Covert-Channels. Evictionbasierte Channels bringen die Mikroarchitektur zunächst in einen bekannten Zustand, um anschließend die Veränderung nach der Ausführung des Senders zu beobachten und entsprechende Informationen abzuleiten. Contentionbasierte Channels encodieren dagegen die Information nicht in einem festen Zustand, sondern in der Stärke der Konkurrenz um geteilte Ressourcen, die sich beispielsweise als Zugriffszeit messen lässt.

Im Kontext von Transient Execution werden häufig evictionbasierte Covert-Channel genutzt, da bei contentionbasierten Kanälen die Ausführung von Sender und Empfänger zeitgleich stattfinden muss und damit nur ein binärer Wert je transient Fenster übertragen werden kann. Dennoch existiert Forschung zu diesen Kanälen, da viele Mitigationen gegen klassische evictionbasierte Covert-Channel hier nicht wirken. (Fustos et al., 2020, S. 8)

2.2.1 Cache als Informationsmedium

Jedes komplexere Programm löst bei seiner Ausführung Prozesse in der Speicherarchitektur aus. In höher abstrahierten Sprachen, werden diese Prozesse nur implizit sichtbar, während bei Assembly diese beispielsweise durch „mov“-Instruktionen ausgelöst werden. Die x86-ISA spezifiziert auch sehr granulare Operationen auf der Mikroarchitektur des Speichers beispielsweise durch die „prefetch“-Instruktion (Oracle, 2020).

Der Zustand, ob eine bestimmte Adresse gecacht oder nicht gecacht ist, kann als Information interpretiert und damit als Covert-Channel betrachtet werden. Da dieser Zustand nicht durch die ISA sichtbar ist, wird die Zugriffszeit auf eine Adresse gemessen und anhand eines zeitlichen Schwellenwerts bestimmt, ob diese Adresse gecacht ist oder nicht. Dieser Schwellenwert kann durch vorherige empirische Messungen in demselben System festgelegt werden. Neben der einfachen Encodierung von Werten durch den Zustand einer Adresse im Cache existieren weitere Low-Level-Covert-Channels, die ebenfalls den Cache oder Elemente der Cachearchitektur nutzen. Ein solcher Ansatz wird in Anhang E erläutert. In dieser Arbeit liegt der Fokus jedoch auf Covert-Channels, bei denen Informationen in den Zustand von Adressen kodiert werden.

Ursprünglich konzentrierten sich Angriffe mit CbCC vor allem auf Side-Channels zum Mitlesen kryptografischer Prozesse zwischen Prozessen und virtuellen Maschinen (Bernstein, 2005, Zhang et al., 2012, Su und Zeng, 2021). Es wurden jedoch auch Covert-Channels mit einem vom Angreifer kontrollierten Sender betrachtet, insbesondere im Kontext von Cross-VM Kommunikation (Xu et al., 2011, Maurice et al., 2017).

Besonders relevant sind CbCC seit Kocher et al., 2019 jedoch für Angriffe mit spekulativer Ausführung, wie in Abschnitt 2.3 erläutert wird.

Das Encodieren erfordert das Schreiben unabhängiger Zuständen in den Cache. Dazu wird ein Speicherbereich allokiert und einzelne Adressen werden als Informationswerte geladen beziehungsweise evicted. Der Abstand zwischen diesen Adressen wird als „Stride“ bezeichnet und muss mindestens der Größe eines Cache-Blocks entsprechen, da sich sonst mehrere Informationsadressen in einem Block befinden und diese damit nicht mehr unabhängig sind. Moderne CPUs implementieren einen Stride-Prefetcher, der Adressen mit einem regelmäßigen Abstand innerhalb einer Memory-Page prefetchen kann (Saileshwar et al., 2021, S. 739). Aus diesem Grund sollte zur Verringerung der Fehlerwahrscheinlichkeit als Stride die Größe einer Memory-Page gewählt werden, um das Prefetchen von naheliegenden Adressen zu verhindern. Diese Beobachtung zeigt sich in Abschnitt 3.3 an praktischen Messungen.

Für CbCC wurden von McIlroy et al., 2019, S. 13 zwei Encodierungen betrachtet: „direct-mapped bits“ (bitweise) und „indexed values“ (Array-Index). Bei der bitweisen Encodierung symbolisiert jede verwendete Adresse ein Informationsbit durch den Zustand, ob die Adresse gecacht oder nicht gecacht ist. Bei der Array-Index-Encodierung wird die Information, als Zahlenwert interpretiert und die Adresse mit diesem Index gecacht. Das bedeutet in der Programmierung, dass $Array + (Stride * Secret)$ in den Cache geladen wird, wobei *Array* die Startadresse des Arrays ist. Die Anzahl der während eines transient Fensters übertragenen Informationen wird in dieser Arbeit in Bit betrachtet. Entsprechend können während eines transient Fenster n Bit übertragen werden, wenn die Encodierung den Cache innerhalb des Fensters in 2^n verschiedene Zustände versetzen kann.

Der größte Vorteil von Array-Index-Encodierung besteht darin, dass die Informationsübertragung grundsätzlich nicht von der Länge des transient Fensters abhängt, da für die Encodierung nur ein Memory Load oder Eviction nötig ist. Der Nachteil besteht darin, dass der bitweise Informationsgehalt einer Übertragung $\log_2 \left(\frac{Array}{Stride} \right)$ beträgt und $\frac{Array}{Stride}$ verwendete Adressen evicted und zeitlich gemessen werden müssen. Bei der bitweisen Encodierung müssen dagegen nur so viele Adressen evicted und gemessen werden, wie Informationsbits übertragen werden. Dafür ist die Anzahl der übertragenen Bits durch die Größe des transient Fensters begrenzt. Eine gewisse Kombination der beiden Konzepte ist auch möglich und wurde in den POCs von Angriffen, wie beispielsweise in Hertogh et al., o. D., realisiert. Dort wurde ein Byte in zweimal 16 Adressen, anstatt in 256 Adressen encodiert. Dafür wurde das zu übertragende Byte in zwei 4-Bit große Teile geteilt und der numerische Wert jeweils als Index in ein Array mit 16 Adressen encodiert. Aus diesem Grund mussten für diese Encodierung allerdings zwei Adressen während der Transient Execution geladen werden, anstatt nur eine.

Die Übertragung durch evictionbasierte CbCC erfolgt wie die meiste Kommunikation in drei Schritten: Vorbereitung, Encodierung und Auslesen. Die Vorbereitung besteht darin das Medium in einen bekannten Zustand zu versetzen. Die Encodierung verändert den Zustand des Mediums und bei dem Auslesen wird der potenziell veränderte Zustand gemessen. Während bei contentionbasierten Covert-Channels die Encodierung und das Auslesen zeitgleich stattfinden, erfolgen diese Phasen bei CbCC sequenziell in der ge-

nannten Reihenfolge. Im Kontext von Transient Execution können diese Phasen sowohl in dem vom Angreifer kontrollierten, als auch im Ziel Kontext stattfinden. Dadurch kann es passieren, dass die beiden Prozesskontexte keine Speicherbereiche teilen und damit ein direkter Zugriff auf geteilte Ressourcen nicht möglich ist. (Xiong und Szefer, 2022, S. 15)

Um dennoch den Cache-Zustand einer Adresse außerhalb des zugreifbaren Speichers zu ermitteln, werden häufig Eviction-Sets verwendet. Dabei kann durch vorherige Messungen bestimmt werden, welche der eigenen Adressen zu einem Eviction-Set anderer Adressen im Ziel Kontext gehören (Vila et al., 2019). Das Laden dieser Adressen im Zielkontext führt zur Eviction einer Adresse im eigenen Adressraum, wenn die physischen Einträge des Eviction-Sets vorher gefüllt wurden (Liu et al., 2015, S. 607).

Störungen bei der Übertragung durch CbCC können durch verschiedene mehr oder weniger beeinflussbare Faktoren auftreten. Auch hier sind die Störungsquellen abhängig von der konkreten Implementierung. Störungen und daraus folgende Fehler wurden besonders bei contentionbasierten CbCC untersucht, wobei die Fehlerquellen nicht identisch zu den möglichen Störungen bei evictionbasierten CbCC sind. Bei CbCC ist ein zentrales Problem die Synchronisierung und die damit verbundenen Timing-Probleme (Maurice et al., 2017, S. 6, 8), was im Kontext von Transient Execution kein Problem darstellt. Andere Störungsquellen sind bei Transient Execution dennoch identisch zu contentionbasierten CbCC zwischen zwei verschiedenen Kontexten. Beispiele hierfür sind das Laden anderer Adressen aus demselben Cache-Block durch dasselbe Programm, zufälliges Laden durch den Prefetcher der CPU oder zufällige Evictions durch das Laden von Adressen im selben Eviction-Set durch dasselbe oder ein anderes Programm. Zusätzlich kann je nach CPU Architektur das Scheduling zu Fehlern führen. Wenn beispielsweise bei der Cache Architektur aus Abbildung 1c der Prozess vor dem Auslesen auf einen anderen Kern verschoben wird, ist es aufgrund des private LLC nicht mehr möglich bei dem Laden der Adresse den encodierten Zustand abzufragen. (Wu et al., 2012, S. 4) Covert-Channel, bei denen sich Sender und Empfänger nicht auf demselben Kern befinden müssen, werden als Cross-Core-Channels bezeichnet.

2.2.2 Realisierung verschiedener Cache-Covert-Channel

Obwohl das Kommunikationsmedium bei CbCC grundsätzlich immer der Zustand einer Adresse ist, also ob diese Adresse gecacht ist oder nicht, kann sich die Methode für das Verändern des Zustands, das Lesen des Zustands und auch die konkret gemessenen Zustände, zwischen verschiedenen Techniken stark unterscheiden.

Viele einfache Cross-Core-CbCC nutzen als Encodierung die unterschiedlichen Zugriffszeiten, je nachdem, ob sich eine Adresse im Cache oder nur im Speicher befindet. Die Methoden zum Verändern beziehungsweise Lesen des Zustands umfassen dabei meist die `clflush`-Instruktion und das Laden oder Schreiben von Adressen. (Yarom und Falkner, 2014, S. 722, Maurice et al., 2015, S. 6)

Diese Art von CbCC wurde bei bisher demonstrierten TEAs häufig verwendet (Xiong und Szefer, 2022, S. 23), auch wenn komplexere Techniken, die besonders die unter-

ren Cacheschichten verwenden, deutlich höhere Bandbreiten erreichen können, teilweise auch bei Cross-Core Kommunikation und ohne geteilte Speicherbereiche. (Maurice et al., 2015, S. 13, H. Wang et al., 2023, S. 10, Rauscher et al., 2025, S. 5)

Wie bereits erwähnt, sind durch Eviction-Sets auch Covert-Channel ohne geteilte Speicherbereiche möglich. Ist ein Eviction-Set gefunden, kann durch gezieltes Laden dieser Adressen eine Eviction ausgelöst und damit der Zustand des Caches in beiden Speicherkontexten verändert werden. Der Angreifer kann anschließend prüfen, ob eine seiner Adressen aus dem Eviction-Set aus dem Cache entfernt wurde und dadurch ermitteln, welche Adresse geladen wurde. Diese Technik wurde, wie viele andere CbCC, ursprünglich als Side-Channel Angriff auf Verschlüsselung angewendet (Osvik et al., 2006, S. 10–13) und später für Cross-Prozess beziehungsweise Cross-VM Covert-Channel Kommunikation (Liu et al., 2015, S. 620) genutzt. Ein Nachteil dieses Covert-Channels ist der Aufwand Eviction-Sets zu finden und diese vor der Übertragung mit Blöcken zu füllen.

Welche CbCC für einen Angriff verfügbar sind, hängt von der Hardware, also der Anzahl und Eigenschaften der Cache-Ebenen, von der ISA sowie von dem Kontext des Ziel- und Angreiferprogramms ab. So wird beispielsweise bei Flush+Reload die Adresse vom Zielkontext in alle Cache-Ebenen geladen. Der Angreiferkontext kann beim Reload die Adresse sowohl aus dem LLC, als auch dem L1- oder L2-Cache laden, je nachdem ob sich der Prozess auf demselben oder einem anderen Prozessor Kern befindet (Xiong und Szefer, 2022, S. 23) und sofern von einem inclusive, private L1- und L2-Cache sowie einem geteilten L3-Cache ausgegangen wird.

Schon durch kleine Änderungen können CbCC je nach Kontext optimiert werden. Ein Beispiel dafür ist Flush+Flush als Abwandlung von Flush+Reload. Anstatt eine Adresse zum Auslesen zu laden, wird die Adresse erneut mit `clflush` aus dem Cache entfernt. Wenn die Adresse gecacht ist, weist diese Instruktion eine leicht höhere Laufzeit auf, die gemessen werden kann. (Gruss et al., 2016, S. 4)

Verschiedene Techniken werden in Anhang F ausführlicher erläutert.

2.3 Transient Execution Angriffe (TEAs)

Optimierungsmechanismen, wie Transient Execution und OoO-Execution, können ausgenutzt werden, um auf der CPU Instruktionen auszuführen, die aus architektonischer Sicht nicht ausgeführt werden sollten. Dabei kann auf Informationen zugegriffen oder abgeleitet werden, die für den jeweiligen Prozesskontext nicht sichtbar sein sollten. (Canella et al., 2019, S. 249, 251)

Ein Angriff umfasst drei Phasen, die in Abbildung 2 dargestellt sind: Vorbereitung, Ausführung und Auslesen. In der Vorbereitungsphase wird der Zustand der Mikroarchitektur in einen gewünschten Zustand versetzt. Beispielsweise werden Speicheradressen für die Übertragung der Information durch den CbCC evictet oder ein Predictor der CPU trainiert. Danach folgt die Phase der Transient Execution, in der das gesuchte Secret ausgelesen und in den Covert-Channel geschrieben, beziehungsweise die Mikroarchitektur verändert wird. Diese Zustandsänderung kann danach gemessen und daraus die übertragene Information abgeleitet werden. Das Ende der Transient Execution führt zum Zurücksetzen des architektonischen Zustands und das Programm setzt seine norma-

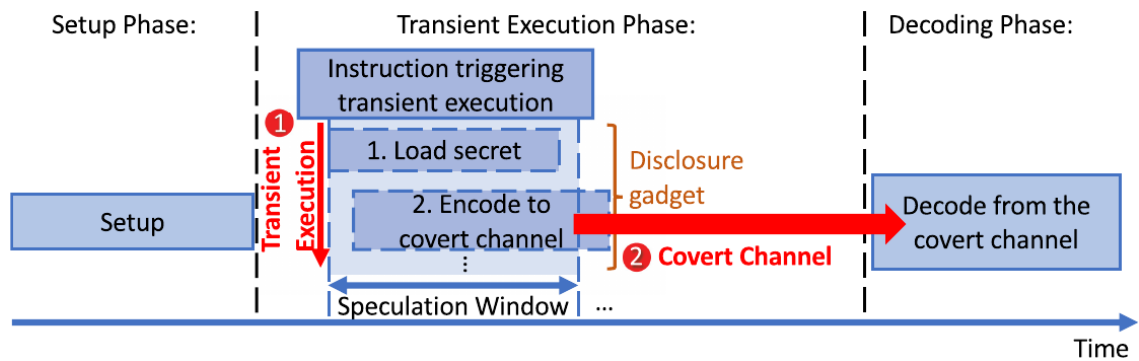


Abbildung 2: Phases of transient execution attacks. (Xiong und Szefer, 2022, S. 2)

le Ausführung fort. Das Auslesen des Secrets, beispielsweise aus dem Cache-Zustand, bringt die Information schließlich in den architektonischen Zustand des Programms. (Xiong und Szefer, 2022, S. 2, Canella et al., 2019, S. 251)

Transient Execution ist dabei nicht garantiert. So können beispielsweise Branch-Instruktionen zwischen den Trainingsläufen den Zustand des Predictors verändern, oder im Falle von dem ursprünglichen Meltdown Angriff kann die Exception eintreten, bevor der Wert geladen wurde (Lipp et al., 2020, S. 9, 10).

2.3.1 Einordnung verschiedener Angriffe

Eine Taxonomie von Angriffen mit Transient Execution wurde von Canella et al., 2019, S. 252–258 vorgeschlagen, ist inzwischen jedoch teilweise veraltet. Grundsätzlich wird dort zwischen Spectre- und Meltdown-ähnlichen Angriffen unterschieden. Meltdown-ähnliche Angriffe nutzen OoO Execution aus, um Instruktionen, die auf einen Programmfehler folgen, dennoch auszuführen. Spectre dagegen nutzt einen Predictor aus, um den Control Flow oder Datenzugriffe zu manipulieren. Beispiele für in dieser Taxonomie noch nicht berücksichtigte Angriffe sind solche, die Load-Adress-Prediction (Kim, Genkin und Yarom, 2025, S. 9) oder Load-Value-Prediction (Kim, Chuang et al., 2025, S. 8, 9) ausnutzen.

Darüber hinaus wird in dieser Einordnung nicht berücksichtigt, ob der Angreifer die transient ausgeführten Instruktionen kontrolliert. Ist dies nicht der Fall, muss die Information - wie bei Kocher et al., 2019, S. 8 - über ein Gadget in den Zustand der Mikroarchitektur geschrieben werden. Solch ein Gadget ist ein Programmabschnitt in dem Zielkontext, bei dem der Cache-Zustand abhängig von dem gesuchten Secret geändert wird. In diesem Falle kann das Auslesen gewissermaßen als Side-Channel-Angriff betrachtet werden, da die transient ausgeführten Instruktionen ursprünglich nicht der Informationsübertragung dienen. Zudem kann die Art der Encodierung und damit auch die Technik zum Auslesen nur eingeschränkt gewählt werden. Aus diesem Grund werden Angriffe, mit Gadgets in dem Zielkontext, in dieser Arbeit nicht weiter erläutert. Kontrolliert der Angreifer dagegen die transient ausgeführten Instruktionen, kann er beliebige Encodierungen und mikroarchitektonische Zustände wählen, wodurch die Übertragung eher einem Covert-Channel entspricht.

2.3.2 Zeitlicher Aufwand von TEAs

Vollständige Angriffe können wenige Sekunden bis mehrere Stunden dauern. Dabei werden viele Übertragungen mit den genannten Phasen ausgeführt, um Speicherbereiche gezielt nach einem Schlüssel zu durchsuchen (Hertogh et al., o. D., S. 7–9) oder einfach einen gewissen Speicherbereich zu übertragen (Kim, Genkin und Yarom, 2025, S. 10). Den größten Zeitaufwand verursachen dabei meistens die Setup- oder Decode-Phase aus Abbildung 2. Der Grund dafür ist, dass die Transient Execution in der Regel aus nur wenigen Instruktionen besteht und die Länge des transient Fensters nur grob über die Art der zugrunde liegenden Abhängigkeit beeinflusst werden kann. Der zeitliche Aufwand der Setup-Phase hängt dagegen sowohl von der Art der Transient Execution als auch vom gewählten CbCC ab. Die Dauer der Decode-Phase ist grundsätzlich nur von dem gewählten CbCC abhängig.

Die Art der Transient Execution wird hauptsächlich von der ausgenutzten Schwachstelle bestimmt und lässt sich größtenteils nur für spezifische Hardware optimieren. Dies kann beispielsweise durch die Reduktion der Trainingsläufe auf ein Minimum passieren. Der CbCC hat jedoch, insbesondere bei Meltdown-Angriffen, den größten Einfluss auf die Gesamtlaufzeit (Lipp et al., 2020, S. 10).

Wie bereits beschrieben, erfolgt die Übertragung durch CbCC in drei Schritten: Vorbereitung, Encodierung und Auslesen. Unabhängig davon, in welchem Prozesskontext diese Phasen ausgeführt werden, findet die Encodierung immer während der Transient Execution statt, da nur zu diesem Zeitpunkt das Secret verfügbar ist. Für die zeitliche Optimierung sind daher insbesondere die Vorbereitung und das Auslesen entscheidend.

Aufgrund der in 2.2.1 beschriebenen Fehler muss die Übertragung des Secrets mehrfach wiederholt werden. Zwar besteht grundsätzlich die Möglichkeit Fehlerkorrekturverfahren in diesem Kontext einzusetzen, dieser Ansatz wurde jedoch in der mir bekannten Literatur bislang nicht betrachtet. Fehlerkorrektur durch Wiederholen des Angriffs und bilden eines Mittelwerts der Ergebnisse wie in Lipp et al., 2020, S. 9 verwendet, ist dennoch nötig, da wie beschrieben die Transient Execution bei TEAs nicht jedes Mal erfolgreich sein muss und dies nicht erkannt werden kann.

Zuletzt stellt sich die Frage der Encodierung. Die beiden möglichen Encodierungen - bitweise oder als Array-Index - beziehungsweise Kombinationen aus beiden, stellen eine Abwägung zwischen der Anzahl an Cache-Zustandsänderungen und der Anzahl übertragener Bits dar.

2.4 Mitigationen von Spectre und CbCC

Mitigationen gegen TEAs auf Hardwareebene versuchen meist Covert-Channels zu verhindern, konzentrieren sich allerdings nur auf eine spezifische Art von CbCC, beispielsweise durch das Verhindern des Ladens von Adressen während Transient Execution. (Yan et al., 2018, S. 2) Das Problem dabei ist, dass potenziell andere Instruktionen, wie `cldemote` in der Zukunft für die Veränderung des Cache-Zustands verwendet werden könnten und generell andere Covert-Channels genutzt werden könnten. (Rauscher et al., 2025, S. 3, 4) Die grundlegendste und garantierte Mitigation, wäre das Entfer-

nen von Transient, und damit OoO und spekulativer Ausführung, aus neuen Prozessoren. Die Folge wäre allerdings eine deutliche Reduzierung der CPU-Performance (Xiong und Szefer, 2022, S. 25), weshalb mir keine in dieser Form eingesetzten Mitigationen bekannt sind. Andere Angriffe wurden häufig durch Mikrocode-Updates mitigiert, wobei diese nicht immer eine vollständige Mitigation garantieren (Schwarz et al., 2019, S. 13). Einige Schwachstellen wurden in dem ausgenutzten Programm für die spezifisch ausgenutzte Software mitigiert, beispielsweise durch das Leeren bestimmter Buffer, vor einem Kontextwechsel, oder erhöhte Speicherisolation. (Lipp et al., 2020, S. 14, Schwarz et al., 2019, S. 13) Das Problem dabei ist, dass auch diese Lösungen zu gewissen Performanceeinbußen führen. Außerdem besteht das ursprüngliche Problem weiter, wodurch neue Angriffe auf Basis derselben Schwachstelle, wie beispielsweise Hertogh et al., o. D., entwickelt werden können.

3 Methodik und Auswertung

Da bisher keine praktische Implementierung der bitweisen Encodierung existiert, besteht das erste Ziel dieser Arbeit darin, zu zeigen, dass eine bitweise Übertragung möglich ist und welche Grenzen sie hat. Dieser experimentelle Abschnitt gibt erste Einblicke in die Betrachtung der Fehlerwahrscheinlichkeiten der beiden Encodierungen sowie in ihren Einfluss auf die Laufzeit. Die danach ermittelte theoretische Grenze betrachtet nicht die Korrektheit einer Übertragung und geht davon aus, dass Übertragungen durch den CbCC immer korrekte Ergebnisse liefern.

3.1 Threat-Model

Um die Möglichkeiten der Optimierung des CbCC einzugrenzen, wird das folgende Threat-Model für TEAs betrachtet:

Der Angreifer hat

- Code-Execution auf dem physischen Zielsystem. (in Sandbox/VM oder als Prozess)
- Zugriff auf Zeitmessungen mit mindestens einer Millisekunde Auflösung.
- einen Angriff, mit dem er während eines transient Fensters Zugriff auf ein gesuchtes Secret hat.
- die Möglichkeit, während des transient Fensters eigenen Code auszuführen.
- Zugriff auf Methoden, um Cache-Zustände mindestens durch das Laden von Adressen zu ändern.
- einen geteilten Speicherbereich mit dem Zielkontext. (z.B. könnte der Zielkontext Zugriff auf physischen Speicher haben)

Verschiedene Angriffe aus der Vergangenheit, die für dieses Threat Model infrage kommen sind Lipp et al., 2020, Maisuradze und Rossow, 2018, Schwarz et al., 2019 Variante 2, Kim, Genkin und Yarom, 2025, Kim, Chuang et al., 2025, Kim et al., 2023, Hertogh et al., o. D. Hierbei sei zu beachten, dass sich der Anwendungsbereich bei fehlendem geteilten Speicherbereich leicht erweitern lässt, da die grundlegenden Konzepte auch für CbCCs gelten, die auf Eviction-Sets basieren, wie z.B. Prime+Probe (Liu et al., 2015). Primär ändert sich der Aufwand durch das Finden von Eviction-Sets, was bereits zu einem Aufwand von unter 100 Millisekunden optimiert wurde (Kessous und Gilboa, 2024, S. 3768), und das Füllen der Cache-Sets während der Setup- beziehungsweise Prime-Phase.

3.2 Methodik der experimentellen Betrachtung

Durch die Literaturrecherche konnten bereits verschiedene Aspekte zur zeitlichen Optimierung von TEAs identifiziert werden. Vorrangig wurde dafür der verwendete Covert-Channel - meist CbCC - identifiziert. Um die Möglichkeiten zur Optimierung und damit

die Grenzen von CbCC im Kontext von Transient Execution zu identifizieren, wurde ein Test-Programm entwickelt, welches die zwei verschiedenen Encodierungen mit verschiedenen Parametern misst. Auf dieser Basis werden die unterschiedlichen Techniken verglichen und die Parameter ermittelt, welche hardwarespezifisch betrachtet werden müssen. Zeitliche Messungen werden mit der `rdtsc`-Instruktion aufgenommen und deshalb in CPU-Zyklen angegeben.

Der Programmablauf des Test-Programms für i Trainingsläufe und n übertragene Bits ist im Folgenden dargestellt:

- | | |
|--|--|
| zeitliche Messung
{
Transient Execution
{ | 1. Ermittlung zeitliche Grenze gecacht/nicht gecacht (tiefer in Anhang G erläutert) |
| | 2. Allokation des Trainingsdaten- ($i + 1$ Elemente) und Covert-Channel-Arrays |
| | 3. Trainings-Array Wert für Index j auf Wert j setzen |
| | 4. Flushen des Training- und Covert-Channel-Arrays (Trainings-Array wird nur Index i geflusht) und Laden des Secrets |
| | 5. Trainieren des Predictors (Ausführen von Transfer mit Index 0 bis $i - 1$) |
| | 6. Prüfung, ob $Array[i] < i$ führt zu spekulativer Ausführung |
| | • Schleife über Anzahl übertragener Bits → Aufbau Bitmaske
• Für bitweise Encodierung jede Iteration laden, bzw. nicht laden des Index
• Für Array-Index-Encodierung, Bitmaske über Secret legen → Index aus Covert-Channel-Array laden (einmalig nach Aufbau der Bitmaske)
• $Array[i] < i$ wird evaluiert, sobald $Array[i]$ geladen → führt zu Rollback (da Wert entsprechend Schritt 3 gesetzt) |
| | 7. Auslesen des Secrets aus Cache-Zustand |
| | 8. Schreiben der Werte in die Ergebnisdatei |

Abbildung 3: Aufbau des Test-Programms

Als Encodierung in den CbCC verwenden die meisten TEAs die Array-Index-Encodierung. (Lipp et al., 2020, Van Schaik et al., 2019, Schwarz et al., 2019, Hertogh et al., o. D., Kim, Genkin und Yarom, 2025, Kim, Chuang et al., 2025) Grund dafür ist zum einen, dass mehr als ein Informationsbit mit dem Laden einer einzelnen Adresse übertragen werden kann, und zum anderen, dass die ursprüngliche Spectre-Beschreibung häufige Muster in Software als Gadgets ausnutzt, bei denen solch ein Speicherzugriff von dem Secret abhängig ist (Kocher et al., 2019, S. 8).

Obwohl bei diesen Angriffen das Gadget aus Code besteht, der vom Angreifer kontrolliert wird, sind mir bisher keine praktischen Versuche in der Literatur bekannt, das Secret binär, mit mehr als einem Bit, zu encodieren. Um die Grenzen von CbCC abzuschätzen, müssten potenzielle Wege, den Durchsatz dieser Kanäle zu erhöhen, gefunden und evaluiert werden. Aus diesem Grund ist ein Ziel dieser Arbeit, Daten binär in einen CbCC

während Transient Execution zu encodieren, um die praktische Umsetzbarkeit zu prüfen und verschiedene technische Eigenschaften zu evaluieren und zu vergleichen.

Als Covert-Channel-Technik wurde in dem Test-Programm der Flush+Reload Covert-Channel (Yarom und Falkner, 2014) verwendet, da das Ziel primär das Demonstrieren von binärer Encodierung in einem evictionbasierten CbCC, nicht die Wahl des Covert-Channels mit dem höchstmöglichen Durchsatz ist. Andere Covert-Channel-Techniken können ein anderes Laufzeitverhalten, sowie andere Fehlerraten und mehr verfügbare Bits aufweisen. Genauso kann für einige Covert-Channel die binäre Encodierung einen größeren oder kleineren Vorteil bieten als für Flush+Reload. Beispielsweise kann die Instruktion für das Encodieren eines Bits eine längere oder kürzere Laufzeit haben und damit mehr oder weniger Bits in einem transient Fenster übertragbar sein.

Sicherstellung von Transient Execution. Um Transient Execution auszulösen, wurde eine Branch-Instruktion verwendet, die außerdem, falls nötig, einen Predictor trainieren kann. Die Funktion, in der sich das transient Fenster befindet, ist in Listing 1 zu sehen. In Abbildung 3 findet dies in Schritt 6 statt. Dort wird abhängig von einem Wert aus einem Array geprüft, ob Daten in den CbCC geschrieben werden sollen oder nicht. Das Array hält für jeden Index i (beginnend mit 0) den Wert $i + 1$, sodass, für alle, außer den letzten Zugriff, die Branch-Bedingung gilt. Das letzte Element der Trainingsdaten wird zudem vor dem Training mit der `clflush`-Instruktion aus dem Cache entfernt, damit die CPU entweder den korrekten Pfad vorhersagen, oder die Ausführung verzögern muss. Während der Trainingsphase, insofern eine angegeben ist, wird als Secret immer 0 in den CbCC geschrieben, damit keine Adressen geladen werden. Wenn von der Methode das Encodieren in den CbCC aufgerufen wird, wird der boolesche Wert „true“ als Indikator für den Trainingsprozess zurückgegeben. In der Übertragungsphase wird „false“ zurückgegeben, um sicherzugehen, dass aus architektonischer Sicht `asm_encode_data` nicht ausgeführt wurde. Wird aus dem CbCC dennoch das Secret gelesen, wurden die Instruktionen als Transient Execution ausgeführt.

Encodierung in den Cache-Zustand. Listing 2 zeigt die Encodierung des Secrets in binärer Form in den Cache-Zustand. In Abbildung 3 findet dies bei den Punkten 6 und 7 statt. Dabei wird ein Array als Eingabe genommen, dessen Elemente vorher durch `clflush` aus dem Cache entfernt wurden. In diesem Array wird mit einem Abstand von `stride` eine Adresse geladen, wenn diese den Wert 1 annehmen soll, ansonsten wird diese übersprungen. Zu beachten sei hier, dass die erste Datenadresse mit einem Abstand von `stride` zum Beginn des Arrays gewählt wurde, um Beeinflussungen des Cache-Zustands durch vorherige Objekte auf dem Stack zu umgehen. Es kann außerdem angegeben werden, wie viele Bits encodiert werden sollen.

Bei der Array-Index-Encodierung wird genauso ein Array übergeben, allerdings wird der Datenwert, nach dem Verrechnen mit einer Bitmaske, mit dem `stride` multipliziert und das Ergebnis als Index für die zu ladende Adresse in dem Array genommen.

Ausgeführt wurde das Programm auf verschiedenen CPUs, von denen die Messwerte in Anhang K zusammengefasst werden.

Ein auffälliger Aspekt ist, dass der CbCC bei einem AMD Ryzen 7 7735HS keine oder nur sehr geringe Daten korrekt transferiert hat. Die Genauigkeit befand sich dauerhaft knapp unter 0,6. Während das Problem die Implementierung des Programms sein könnte, weist der Covert-Channel auf der AMD A10-7850K APU deutlich höhere Genauigkeiten auf. Dies wirft die Frage auf, ob neuere CPUs, wie der Ryzen 7 7735HS, Mitigationen gegen CbCC aufweisen. Da mir keine Quelle bekannt ist, die solch eine Mitigation bei diesen CPUs beschreibt, scheint dies eine offene Forschungsfrage zu sein. Es stellt sich die Frage, ob neuere Ryzen-CPU's Mitigationen gegen CbCC, möglicherweise nur während Transient Execution, besitzen, und wenn dies der Fall ist, welche Arten von Angriffen dadurch verhindert werden. Eine vollständige Mitigation von TEAs durch das Verhindern von Covert-Channels ist zu bezweifeln, da contentionbasierte Covert-Channels nahezu unvermeidbar sind und die einzige bekannte garantierte Mitigation von TEAs das Deaktivieren von Transient Execution ist. Die daraus resultierenden Performanceeinbußen wären zu groß, um die Verhinderung dieser Angriffe zu rechtfertigen.

3.3 Auswertung

Die Abbildungen 6, 7, 8 und 9 zeigen die Ergebnisse als Graphen zusammengefasst.

In diesen Grafiken bedeutet „bitweise korrekt“, dass das Verhältnis aller korrekt übertragenen Bits, über alle Übertragungen hinweg gegenüber der Anzahl an übertragenen Bits ins Verhältnis gesetzt wird. Gesamte Übertragungen korrekt setzt dagegen die Anzahl an Übertragungen, bei denen kein Fehler aufgetreten ist, gegenüber der Anzahl an Übertragungen ins Verhältnis. Nötig ist dies, da bei der bitweisen Encodierung zwar seltener alle Bits korrekt übertragen werden, allerdings eine einzelne Änderung des Cache-Zustands, nicht alle Informationen stört. Bei der Array-Index-Encodierung dagegen wird bei jeglicher Störung keine Information korrekt übertragen. Die Fehlerkorrektur bei der bitweisen Encodierung, durch Wiederholung der Übertragung ist also generell besser möglich. Bei den beiden Darstellungen sei zu beachten, dass bei keinem Signal, das bedeutet bei rein zufällig erhaltenen Werten, die Genauigkeit bitweise betrachtet gegen 0,5 geht, während bei der Betrachtung der korrekten Übertragungen die Genauigkeit gegen 0 geht. Die Skalen sind also nicht identisch zu interpretieren.

Die Abbildungen 8 und 9 zeigen außerdem die durchschnittliche Laufzeit einer Übertragung, gemessen über den in Abbildung 3 gezeigten Abschnitt.

Die Fehlerwahrscheinlichkeit zwischen den beiden Encodierungen verhält sich ähnlich in einem Bereich von 0,8 bis 0,95 über alle Trainingslängen hinweg. Die Array-Index-Encodierung weist grundsätzlich eine höhere Genauigkeit auf, wobei die Genauigkeit der einzelnen Bits nahezu identisch mit der Genauigkeit der Übertragungen übereinstimmt und nicht von der Anzahl übertragener Bits, sondern nur von dem Stride abhängt. Bei der bitweisen Encodierung zeigt sich dagegen, besonders bei dem i7-6700K, dass die

Genauigkeit ab 9 Bit der vollständig korrekten Übertragungen, deutlich abfällt. Dies weist darauf hin, dass bei diesem Prozessor, die maximale Anzahl an `prefetch0`-Instruktionen, die während der Transient Execution ausgeführt werden, 8 beträgt.

Bei der bitweisen Encodierung ist ein großer Vorteil, dass mehr unabhängige Datenpunkte übertragen werden, als bei der Array-Index-Encodierung. Dadurch kann der häufigste Wert für jedes Bit einzeln, anstatt für den Gesamtwert, ermittelt werden. Dadurch könnte bei schlechterer Genauigkeit, verglichen mit der Array-Index-Encodierung, mit derselben Anzahl an Wiederholungen der korrekte Wert übertragen werden. Dafür müssen allerdings alle Bits gleich wahrscheinlich einen Fehler aufweisen. Im Folgenden wird davon ausgegangen, dass Fehler bei allen Bits gleich wahrscheinlich sind.

Verschiedene Aspekte der Messungen zeigen, dass eine Generalisierung der Fehlerwahrscheinlichkeit abhängig von gegebenen Parametern schwer ist. Die Ergebnisse der AMD A10-7850K APU zeigen bei der bitweisen Encodierung für ein bis drei encodierte Bit deutlich geringere Genauigkeiten als für höhere Bitzahlen. Bei der Array-Index-Encodierung dagegen ist bei dieser CPU für ein bzw. zwei encodierte Bits die höchste Genauigkeit aller Übertragungen verzeichnet. Auffällig ist dabei auch, dass ab einem Stride von 2^{10} die Genauigkeit nur noch sinkt. Während der beobachtete Unterschied zwischen der bitweisen und der Array-Index-Encodierung von dem Unterschied in der Encodierung abhängt, wird in dieser Arbeit kein Versuch unternommen, diese beobachteten Phänomene zu erklären. Sie zeigen aber, welche Problematik die Betrachtung der Fehlerwahrscheinlichkeit für die Ermittlung des Durchsatzes darstellt.

Der Stride hat wie erwartet einen deutlichen Einfluss auf die Genauigkeit. Interessant ist jedoch, dass bei dem Intel® Core™ i7-6700K ab einem Stride von 2^9 kaum eine Verbesserung der Genauigkeit zu erkennen ist. Auch bei den anderen CPUs zeigt sich ein ähnliches Verhalten, was auf die maximale Reichweite des Prefetchers hindeutet. Bei einem Stride von 2^{12} , was der Größe einer Memory-Page entspricht, lassen sich jedoch zuverlässig die besten Ergebnisse beobachten.

Die Anzahl der Trainingsläufe hat, wie in Abbildung 7 zu sehen, bei der Array-Index-Encodierung keinen relevanten Einfluss auf die Genauigkeit der Übertragung. Bei der bitweisen Encodierung dagegen ist eine leicht steigende Tendenz der Genauigkeit bei steigender Anzahl an Trainingsläufen zu erkennen. Eine mögliche Erklärung dafür wäre, dass mit mehr Trainingsläufen die Länge des transient Fensters steigt und damit auch die Anzahl an korrekt übertragenen Bits bei der bitweisen Encodierung. Bei der Array-Index-Encodierung dagegen ändert die Größe des transient Fensters nichts mehr, sobald dessen Dauer für die Encodierung einer Adresse ausreicht.

Das Laufzeitverhalten ist als logarithmische Skala dargestellt, da die Laufzeit der Array-Index-Encodierung mit zunehmender Bitanzahl exponentiell skaliert. Grund dafür ist, dass, wie bereits erwähnt, die Anzahl der zu messenden Adressen exponentiell mit der Anzahl an übertragenen Bit steigt.

Die Abbildungen 8 und 9 zeigen, dass der Stride einen Einfluss auf die Laufzeit der Übertragung hat. Die Ursache dafür ist, dass in dem Test-Programm die Allokation von dem Covert-Channel-Array sowie das Setzen der Array-Elemente auf einen festen Wert mitgemessen werden und deren Laufzeit von der Größe des Arrays abhängt. Das Setzen von Werten in dem Array ist nötig, da ansonsten bei einem Stride von 4096 die Genauigkeit deutlich sinkt. Der Grund dafür könnte mit der Handhabung von Memory-Pages zusammenhängen, da sich bei diesem Stride nur eine betrachtete Adresse je Page befindet. In einem Angriff mit mehreren Übertragungen kann dasselbe Array mehrfach verwendet werden. Aus diesem Grund ist die Laufzeit dieser Operationen nicht für das Abschätzen einer Grenze für die Laufzeit von TEAs relevant. Für die bitweise Encodierung zeigt sich deutlich, dass der Covert-Channel kaum einen Einfluss auf die Laufzeit einer Übertragung hat. Dies zeigt sich, da nur ein geringer Anstieg der Laufzeit einer Übertragung bei mehr Bit erfolgt. Bei der Array-Index-Encodierung dagegen steigt die Gesamtlaufzeit mit steigender Bitanzahl deutlich an. Dies deutet bereits darauf hin, dass die optimale Encodierung eine Kombination der beiden ist.

3.4 Theoretische Laufzeitbetrachtung

Im Folgenden wird die Laufzeit einer Übertragung während eines TEA unter einigen Annahmen und ohne die Genauigkeit des CbCC zu beachten betrachtet. Dabei wird als Zeiteinheit immer implizit CPU-Zyklen verwendet.

Folgende Annahmen werden getroffen:

- Die ausgeführten Operationen dauern bei gleichem Zustand der Mikroarchitektur immer exakt gleich lang, gemessen an den CPU-Zyklen.
- Alle möglichen übertragenen Informationen sind gleich wahrscheinlich.
- Der Unterschied der Dauer des Auslesens einer Adresse, wenn diese gecacht ist, gegenüber der Laufzeit, wenn diese nicht gecacht ist, wird nicht unterschieden. Für die bitweise Encodierung kann zwar, unter der Annahme, dass ein Informationsbit mit einer Wahrscheinlichkeit von $\frac{1}{2}$ den Wert 1 hat, der Mittelwert der beiden verwendet werden, allerdings gilt dies nicht für die Array-Index-Encodierung. Als Zugriffszeit, nachfolgend T_R , wird die kürzere der beiden Zeiten betrachtet, um eine obere Grenze für den Durchsatz durch Überschätzen zu ermitteln.
- Eine Übertragung kann beliebig oft wiederholt werden, um das Secret schrittweise zu übertragen.
- Nach dem Auslesen der letzten Übertragung und der nächsten Setup-Phase vergeht keine Zeit.

Die Setup-Phase muss in der Regel eine Operation, wie das Evicten von Adressen für alle beobachteten Adressen durchführen. Zusätzlich muss der Predictor, sofern einer verwendet wird, trainiert werden. Damit ergibt sich die Laufzeit L_S mit N_T als Anzahl der

Trainingsläufe, T_T als die Dauer eines Trainingslaufs, N_P als die Anzahl der beobachteten Adressen und T_P als Dauer der Vorbereitung einer Adresse.

$$L_S = N_T * T_T + N_P * T_P$$

Die Vorbereitung einer Adresse besteht beispielsweise bei Flush+Reload in der Dauer einer Flush-Instruktion.

Für die Transient-Execution-Phase wird der zeitliche Aufwand L_E als konstanter Wert betrachtet, da der Wert vorrangig von der Kondition abhängt, die das transient Fenster auslöst.

In der Decode-Phase hängt die Laufzeit primär von dem verwendeten Covert-Channel ab. Diese Laufzeit L_D hängt von der Anzahl auszulesender Adressen N_R und dem zeitlichen Aufwand zum Auslesen einer Adresse T_R ab.

$$L_D = N_R * T_R$$

Bei der Setup- und der Decode-Phase wird bewusst zwischen N_P und N_R unterschieden, da die Anzahl der vorbereiteten Adressen zwangsläufig alle möglichen Zustandsadressen umfasst. Bei dem Auslesen kann allerdings für die Array-Index-Encodierung bereits die erste Adresse, die als gecacht ermittelt wird, auch als encodierter Wert betrachtet werden. Um die Berechnung der Anzahl der zu messenden Adressen zu vereinfachen, werden alle Werte, die in den CbCC geschrieben werden könnten, als gleich wahrscheinlich angenommen. Daraus folgt, dass im Mittel nach der Hälfte der Adressen der encodierte Wert gefunden wird. Für die bitweise Encodierung ändert sich dadurch nichts, da dort jede Adresse einen Informationswert speichert und somit stets alle Adressen ausgelesen werden müssen.

Die Anzahl an verwendeten Adressen wird für eine verallgemeinerte Kombination aus bitweiser und Array-Index-Encodierung betrachtet. Es werden zur Encodierung Adressen aus n_b Arrays geladen. Jedes Array encodiert 2^{n_a} verschiedene Zustände. Der Wert 0 wird encodiert, indem keine Adresse geladen ist. Daraus ergeben sich für jedes Array $2^{n_a} - 1$ Adressen. n_a stellt dabei die Anzahl an Array-Index encodierten Adressen dar. Damit ergeben sich für N_P und N_R folgende Berechnungen:

$$N_P = n_b * (2^{n_a} - 1)$$

$$N_R = n_b * 2^{n_a-1}$$

Wobei n_b und n_a mindestens 1 sind. Die vollständige bitweise Encodierung liegt also vor, wenn $n_a = 1$ und n_b die Anzahl an übertragenen Bits angibt. Wenn $n_b = 1$ und n_a beliebig, liegt die Array-Index-Encodierung vor. Damit wird davon ausgegangen, dass bei der Array-Index-Encodierung der Wert 0 übertragen wird, indem keine andere Adresse gecacht ist.

Die theoretische Laufzeit L_G einer einzelnen Übertragung mit den benannten Faktoren beträgt somit:

$$L_G = L_S + L_E + L_D$$

$$L_G = N_T * T_T + n_b * (2^{n_a} - 1) * T_P + L_E + n_b * 2^{n_a-1} * T_R$$

Da für einen spezifischen TEA die Transient Execution und deren Vorbereitung gleichbleiben, gilt für hinreichend große n_b , n_a , T_P und T_R :

$$L_G \propto n_b * (2^{n_a} - 1) * T_P + n_b * 2^{n_a-1} * T_R$$

Begrenzt ist n_b durch die Tiefe des transient Fensters und n_a durch die Größe des Cache, beziehungsweise des Speichers.

Der Durchsatz D , welcher optimiert werden soll, ergibt sich aus der Anzahl übertragener Bits n je Zeiteinheit.

$$D = \frac{n}{L_G} = \frac{n_a * n_b}{L_G}$$

Da L_G durch n_b linear und durch n_a exponentiell skaliert, sollte n_b maximal und n_a minimal gesetzt werden, wobei n_b durch das transient Fenster begrenzt ist. Unter der theoretischen Annahme, dass n_b nicht begrenzt ist, ergibt sich, da $N_T * T_T$ und L_E konstant sind, für D bei $n_a = 1$ die folgende theoretische Grenze.

$$\lim_{n_b \rightarrow +\infty} D = \lim_{n_b \rightarrow +\infty} \frac{n_b}{L_G} = \lim_{n_b \rightarrow +\infty} \frac{n_b}{N_T * T_T + n_b * T_P + L_E + n_b * T_R} = \frac{1}{T_P + T_R}$$

Diese Grenze ergibt sich auch trivial durch die Überlegung, welche Operationen bei der bitweisen Encodierung je Bit ausgeführt werden müssen. Unter der Vernachlässigung des zeitlichen Aufwands für das Verändern des Cache-Zustands während der Transient Execution ergibt sich, dass der Zeitaufwand für die Übertragung eines Bits die Summe aus der Vorbereitung und dem Auslesen ist. Folglich ergibt sich der Durchsatz $\frac{1}{T_P + T_R}$.

Da allerdings n_b begrenzt ist, $N_T * T_T$ und L_E nicht beide 0 sind, ergibt sich für einen durch das transient Fenster begrenzten Wert n_{bmax} die folgende Funktion für D .

$$D(n_a) = \frac{n_{bmax} * n_a}{N_T * T_T + L_E + n_{bmax} * T_P * (2^{n_a} - 1) + n_{bmax} * T_R * 2^{n_a-1}}$$

Als Visualisierung ist für $N_T = 50$, $T_T = 20$, $L_E = 100$, $T_P = 50$, $T_R = 120$ und $n_b = 8$ die Funktion $D(n_a)$ in Abbildung 4 dargestellt.

Das Maximum dieser Funktion ergibt sich aus dem Punkt, an dem für die Ableitung $D'(n_a) = 0$ gilt. Die Ableitung ist für Vollständigkeit in Abbildung 5 dargestellt.

3.5 Diskussion

Die Wahl des Strides ist für die Datenrate des CbCC eher weniger relevant. Grund dafür ist, dass ein größerer Stride keine direkten negativen Auswirkungen auf die Datenrate oder Genauigkeit hat und nur durch die Größe des RAM begrenzt ist. Relevant werden könnte dieser Parameter, wenn das allokierte Array so groß ist, dass zu viele beobachtete Adressen in einem Eviction-Set liegen. Beispielsweise, wenn auch der LLC nur wenige

Einträge je Eviction-Set aufweist, die Array-Index-Encodierung verwendet wird und eine Art invertierte Encodierung verwendet wird. Also, wenn alle verwendeten Adressen in der Setup-Phase geladen und während der Transient Execution zur Encodierung einzelne Adressen evicted werden.

Da n_b maximiert werden soll, wäre es sinnvoll, die Laufzeit der Instruktion zum Verändern des Cache-Zustands während der Transient Execution zu minimieren und die Encodierung mit möglichst wenigen Instruktionen zu realisieren. Dadurch könnten möglichst viele Bits encodiert werden. Besonders die Genauigkeit der Zeitmessung beeinflusst die optimale Encodierung. Denn ansonsten muss der Zeitunterschied, wie beispielsweise in Kim, Genkin und Yarom, 2025, S. 17, verstärkt werden. Diese Verstärkung erhöht allerdings auch die Laufzeit einer Messung, da der Unterschied groß genug sein muss, um von dem ungenauen Zeitgeber unterschieden zu werden.

Für viele Angriffe, die darauf abzielen, Zugriffsbeschränkungen, beispielsweise einer VM oder des Kernels, zu umgehen, ergibt es deshalb Sinn, eine Kombination aus bitweiser und Array-Index-Encodierung zu verwenden. Dort ist häufig eine hochauflösende Zeitmessung, wie zum Beispiel durch die `rdtsc`-Instruktion, möglich. Dadurch bleibt die Laufzeit des Auslesens relativ gering im Verhältnis zu der restlichen Übertragung. Beispiele dafür sind Hertogh et al., o. D., Lipp et al., 2020, Maisuradze und Rossow, 2018.

Bei gewissen Angriffen sind allerdings keine hochauflösenden Zeitmessungen möglich. Dies ist beispielsweise in Apple Webkit der Fall (Kim, Genkin und Yarom, 2025, S. 11). Zum Auslesen der Daten aus dem Covert-Channel wurde von Kim, Genkin und Yarom, 2025 der Zeitunterschied zwischen einer gecachten und einer nicht gecachten Adresse durch logische Gatter verstärkt. Das Auslesen einer Adresse dauert im Mittel deshalb zwischen 5 und 12 Millisekunden (Kim, Genkin und Yarom, 2025, S. 17). Dadurch ist T_R im Verhältnis zum restlichen Angriff so groß, dass die Dauer einer Übertragung nahezu der Laufzeit des Auslesens entspricht. Das Verwenden der bitweisen Encodierung verringert die Laufzeit damit deutlich gegenüber der Array-Index-Encodierung. Mit der Annahme, dass mit jeder Übertragung 8 Bit encodiert werden, ergibt sich das folgende Verhältnis zwischen den bitweise encodierten Adressen N_{Ra} und den Array-Index encodierten Adressen N_{Rb} :

$$\frac{N_{Ra}}{N_{Rb}} = \frac{2^{n_a-1}}{n_b} = \frac{128}{8} = 16$$

Da das Auslesen praktisch die Laufzeit der gesamten Übertragung darstellt, wäre in diesem Falle der gesamte Angriff um den Faktor 16 schneller. Dies ist allerdings nur ein theoretischer Wert und gilt nur für Angriffe, bei denen das Auslesen eine sehr hohe Laufzeit im Verhältnis zu der restlichen Übertragung aufweist. Aus Listing 3 und 4 in Kim, Genkin und Yarom, 2025, S. 6, 8 lässt sich ablesen, dass ein Byte als Array-Index encodiert wurde. Die Laufzeit dieses spezifischen Angriffs könnte also unter den getroffenen Annahmen deutlich optimiert werden und ein entsprechend höheres Risiko darstellen. Der dort genannte Durchsatz von 0,384 Bits pro Sekunde (Kim, Genkin und Yarom, 2025, S. 12) könnte theoretisch auf 6,144 erhöht werden.

Das Risiko, das von Angriffen ausgeht, verändert sich mit der Geschwindigkeit, in der

sie durchgeführt werden können. Auch wenn der Durchsatz deutlich erhöht und damit die Laufzeit des zuvor beschriebenen POC deutlich verringert werden kann, bleiben TEAs jedoch aufwändige Angriffe mit hoher Komplexität. Die Optimierung der verwendeten CbCCs trägt zwar dazu bei, die praktische Umsetzbarkeit solcher Angriffe zu erhöhen, allerdings ist für die Einschätzung nach wie vor vorrangig die ausgenutzte Schwachstelle und ihre Auswirkungen auf die Schutzziele relevant. Da aber eine generelle Mitigation für TEAs nicht existiert, bleiben diese Angriffe zukünftig ein Risiko, welches auch von der Laufzeit abhängt. Die ermittelte Grenze für die Laufzeit eines Angriffs kann bei der Einschätzung von Angriffen hinsichtlich der praktischen Ausnutzung helfen. Zusätzlich können Maßnahmen, mit dem Ziel, Covert-Channels während TEAs zu begrenzen, hinsichtlich ihrer Wirkung gemessen und verglichen werden.

4 Fazit

4.1 Ergebnisse

Diese Arbeit betrachtet CbCCs in dem Kontext von TEAs losgelöst von den tatsächlichen Angriffen. Dadurch wird allgemeiner das Problem der Laufzeit der Übertragung von Daten während der Transient Execution analysiert.

Es wird das erste Mal die praktische Umsetzung der bitweisen Encodierung, im Kontext von TEAs, demonstriert und mit der Array-Index-Encodierung verglichen. Dabei zeigt sich, dass die bitweise Encodierung, bezüglich der Anzahl der encodierten Bits, begrenzt ist. Die Array-Index-Encodierung weist dagegen ein exponentiell steigendes Laufzeitverhalten bei steigender Bitanzahl auf. Aus diesem Grund wird der Durchsatz abhängig von der Bitanzahl und verschiedenen messbaren Parametern modelliert und eine Funktion für die Berechnung der optimalen Encodierung hergeleitet.

Des Weiteren wird die obere Grenze $\frac{1}{T_P+T_R}$ für den Durchsatz von CbCCs im Kontext von Transient Execution ermittelt. Die ermittelte Durchsatzfunktion $D(n_a)$ kann zudem auch als obere Grenze für den Durchsatz eines CbCC interpretiert werden, insofern für T_R die geringere Laufzeit zwischen einer gecachten und einer nicht gecachten Adresse verwendet wird. Zuletzt wird an einem bereits bekannten POC verdeutlicht, wie stark ein praktischer TEA durch die Optimierung des verwendeten CbCC und damit das ausgehende Risiko verändert werden kann.

Auch wenn Gadgets behoben und Mitigationen gegen TEAs in Mikrocode entwickelt werden, bleibt bei bestehender Hardware das zugrunde liegende Problem der Schwachstellen bestehen (Hertogh et al., o. D., S. 3, 10). Neue CPUs verwenden außerdem weiter spekulative Ausführung, um die Performance weiter zu steigern. Dabei handelt es sich nicht nur um Branch-Prediction, sondern auch um neue Predictoren (Kim, Genkin und Yarom, 2025, S. 5). Diese neuen Performanceverbesserungen bieten auch neue Angriffsflächen für TEAs. Diese Art von Angriffen wird also in Zukunft bestehen bleiben und kann präventiv nur begrenzt werden. Aus diesem Grund ist besonders für Endgeräte die Laufzeit eines möglichen Angriffs relevant. Die hergeleitete Funktion für den Durchsatz $D(n_a)$ ermöglicht die Einschätzung der Laufzeit dieser Angriffe unabhängig von der ausgenutzten Schwachstelle.

4.2 Ausblick

Die theoretische Betrachtung des Durchsatzes beachtet nicht die Fehlerrate des CbCC. Besonders in Bezug auf die beiden Encodierungen müsste für einen realistischen Angriff und dessen Limits auch die Korrektheit der übertragenen Daten und damit auch Fehlerkorrekturverfahren in Betracht gezogen werden. Wie die Sektion 3.3 zeigt, verhält sich die Fehlerwahrscheinlichkeit der beiden Encodierungen aufgrund der gegebenen Eigenschaften sehr unterschiedlich. Dies erschwert die Betrachtung der Fehlerwahrscheinlichkeiten für die verallgemeinerte Encodierung deutlich.

Eine weitere offene Frage ist, wie Cache-Zustände optimal während der Transient Execution geändert werden können. Bekannt ist beispielsweise nicht, ob andere Instruk-

tionen, wie `clflush`, in demselben transient Fenster häufiger ausgeführt werden könnten als die `prefetch0`-Instruktion. Des Weiteren könnte die Anzahl der Instruktionen für das Encodieren optimiert werden, um möglichst viele Zustände zu ändern.

Die aktuelle Berechnung für den Durchsatz $D(n_a)$ kann vermutlich durch das Hinzufügen weiterer Parameter präzisiert werden. Dafür könnte beispielsweise ein Parameter für die Zeit zwischen zwei Übertragungen hinzugefügt oder die unterschiedliche Laufzeit zwischen einer gecachten und einer nicht gecachten Adresse beachtet werden. Dadurch könnte der maximale Durchsatz weiter eingegrenzt werden. Letztendlich sollten mathematische Modelle empirisch verifiziert werden, wofür die Bearbeitungszeit für diese Arbeit nicht ausgereicht hat. An einem entsprechenden Programm zur Ermittlung der nötigen Parameter und zum Messen des Durchsatzes für gegebene n_b und n_a habe ich bereits zum Zeitpunkt der Abgabe dieser Arbeit gearbeitet. Die Entwicklung des Programms setze ich fort, mit dem Ziel, empirische Werte des Durchsatzes mit der theoretischen Grenze sowie der Durchsatzfunktion zu vergleichen.

Literatur

- Bernstein, D. J. (2005). Cache-timing attacks on AES.
- Brotzman, R., Zhang, D., Kandemir, M. T., & Tan, G. (2021). SpecSafe: Detecting cache side channels in a speculative world. *Proceedings of the ACM on Programming Languages*, 5, 1–28. <https://doi.org/10.1145/3485506>
- Canella, C., Schwarz, M., & Lipp, M. (2019). A systematic evaluation of transient execution attacks and defenses. <https://doi.org/10.5555/3361338.3361356>
- Cheng, X., Tong, F., Wang, H., Zhou, Z., Jiang, F., & Mao, Y. (2024). SpecLFB: Eliminating cache side channels in speculative executions, 631–646. Zugriff 27. April 2026 unter <https://www.usenix.org/conference/usenixsecurity24/presentation/cheng-xiaoyu>
- CLDEMOTE — *Cache Line Demote*. (2023). Zugriff 24. April 2026 unter <https://www.felixcloutier.com/x86/cldemote>
- Fustos, J., Bechtel, M., & Yun, H. (2020, 23. Juni). SpectreRewind: Leaking secrets to past instructions. <https://doi.org/10.48550/arXiv.2003.12208>
- Gruss, D., Maurice, C., Wagner, K., & Mangard, S. (2016). Flush+flush: A fast and stealthy cache attack [Series Title: Lecture Notes in Computer Science]. In J. Caballero, U. Zurutuza & R. J. Rodríguez (Hrsg.), *Detection of intrusions and malware, and vulnerability assessment* (S. 279–299, Bd. 9721). Springer International Publishing. https://doi.org/10.1007/978-3-319-40667-1_14
- Handy, J. (1993). *The cache memory book*. Academic press.
- Hat, U. D. R. (2007). What every programmer should know about memory.
- Hertogh, M., Quakkelaar, D., Raymakers, T., Sarma, M. H., Muench, M., & Bos, H. (o. D.). Rain: Transiently leaking data from public clouds using old vulnerabilities. *Vorveröffentlichung*.
- Kaeli, D., & Yew, P.-C. (Hrsg.). (2005, 26. Mai). *Speculative execution in high performance computer architectures* (0. Aufl.). Chapman; Hall/CRC. <https://doi.org/10.1201/9781420035155>
- Kamp, P.-H., & Watson, R. (2004). Building systems to be shared, securely: Want to securely partition VMs? one option is to put 'em in jail. *ACM Queue*, 2(5), 42–51. <https://doi.org/10.1145/1016998.1017001>
- Kessous, T., & Gilboa, N. (2024). Prune+PlumTree - finding eviction sets at scale. *2024 IEEE Symposium on Security and Privacy (SP)*, 3754–3772. <https://doi.org/10.1109/SP54263.2024.00173>
- Kim, J., Chuang, J., Genkin, D., & Yarom, Y. (2025). FLOP: Breaking the apple m3 CPU via false load output predictions.
- Kim, J., Genkin, D., & Yarom, Y. (2025). SLAP: Data speculation attacks via load address prediction on apple silicon.
- Kim, J., Van Schaik, S., Genkin, D., & Yarom, Y. (2023). iLeakage: Browser-based timerless speculative execution attacks on apple devices. *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2038–2052. <https://doi.org/10.1145/3576915.3616611>

- Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., & Yarom, Y. (2019). Spectre attacks: Exploiting speculative execution. *2019 IEEE Symposium on Security and Privacy (SP)*, 1–19. <https://doi.org/10.1109/SP.2019.00002>
- Lampson, B. W. (1973). A note on the confinement problem. *Communications of the ACM*, 16(10), 613–615. <https://doi.org/10.1145/362375.362389>
- Liedtke, T. (2022). *Informationssicherheit: Möglichkeiten und Grenzen*. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-662-63917-7>
- Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., Hamburg, M., & Strackx, R. (2020). Meltdown: Reading kernel memory from user space. *Communications of the ACM*, 63(6), 46–56. <https://doi.org/10.1145/3357033>
- List of x86 instructions [Page Version ID: 1351002386]. (2026, 25. April). In *Wikipedia*. Zugriff 27. April 2026 unter https://en.wikipedia.org/w/index.php?title=List_of_x86_instructions&oldid=1351002386
- Liu, F., Yarom, Y., Ge, Q., Heiser, G., & Lee, R. B. (2015). Last-level cache side-channel attacks are practical. *2015 IEEE Symposium on Security and Privacy*, 605–622. <https://doi.org/10.1109/SP.2015.43>
- Maisuradze, G., & Rossow, C. (2018). Ret2spec: Speculative execution using return stack buffers. *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2109–2122. <https://doi.org/10.1145/3243734.3243761>
- Mandl, P. (2020). *Grundkurs Betriebssysteme: Architekturen, Betriebsmittelverwaltung, Synchronisation, Prozesskommunikation, Virtualisierung*. Springer Fachmedien Wiesbaden. <https://doi.org/10.1007/978-3-658-30547-5>
- Maurice, C., Neumann, C., Heen, O., & Francillon, A. (2015). C5: Cross-cores cache covert channel [Series Title: Lecture Notes in Computer Science]. In M. Almgren, V. Gulisano & F. Maggi (Hrsg.), *Detection of intrusions and malware, and vulnerability assessment* (S. 46–64, Bd. 9148). Springer International Publishing. https://doi.org/10.1007/978-3-319-20550-2_3
- Maurice, C., Weber, M., Schwarz, M., Giner, L., Gruss, D., Boano, C. A., Mangard, S., Roemer, K., & Mangard, S. (2017). Hello from the other side: SSH over robust cache covert channels in the cloud. *Proceedings 2017 Network and Distributed System Security Symposium*. <https://doi.org/10.14722/ndss.2017.23294>
- McIlroy, R., Sevcik, J., Tebbi, T., Titzer, B. L., & Verwaest, T. (2019, 14. Februar). Spectre is here to stay: An analysis of side-channels and speculative execution. <https://doi.org/10.48550/arXiv.1902.05178>
- Mosier, N., Nemati, H., Mitchell, J. C., & Trippel, C. (2025, 22. Januar). Analyzing and exploiting branch mispredictions in microcode. <https://doi.org/10.48550/arXiv.2501.12890>
- Oracle. (2020). *PREFETCH Instructions - x86 Assembly Language Reference Manual*. Zugriff 11. April 2026 unter https://docs.oracle.com/cd/E37838_01/html/E61064/gpjfg.html

- Osvik, D. A., Shamir, A., & Tromer, E. (2006). Cache attacks and countermeasures: The case of AES [Series Title: Lecture Notes in Computer Science]. In D. Pointcheval (Hrsg.). D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi & G. Weikum (Bearb.), *Topics in cryptology – CT-RSA 2006* (S. 1–20, Bd. 3860). Springer Berlin Heidelberg. https://doi.org/10.1007/11605805_1
- Paccagnella, R., Luo, L., & Fletcher, C. W. (2021). Lord of the ring(s): Side channel attacks on the CPU on-chip ring interconnect are practical.
- Rauscher, F., Fiedler, C., Kogler, A., & Gruss, D. (2025). A systematic evaluation of novel and existing cache side channels. *Proceedings 2025 Network and Distributed System Security Symposium*. <https://doi.org/10.14722/ndss.2025.230253>
- Saileshwar, G., Fletcher, C. W., & Qureshi, M. (2021). Streamline: A fast, flushless cache covert-channel attack by enabling asynchronous collusion. *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 1077–1090. <https://doi.org/10.1145/3445814.3446742>
- SARANGI, S. R. (2021). *BASIC COMPUTER ARCHITECTURE* [OCLC: 1273677925]. WHITE FALCON PUBLISHING.
- Schwarz, M., Lipp, M., Moghimi, D., Van Bulck, J., Stecklina, J., Prescher, T., & Gruss, D. (2019). ZombieLoad: Cross-privilege-boundary data sampling. *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 753–768. <https://doi.org/10.1145/3319535.3354252>
- Server processor specifications* [AMD]. (o. D.). Zugriff 4. April 2026 unter <https://www.amd.com/en/products/specifications/server-processor.html>
- Shepherd, M., Brookes, S., & Denz, R. (2022). Transient execution and side channel analysis: A vulnerability or a science experiment? *International Conference on Cyber Warfare and Security*, 17(1), 288–297. <https://doi.org/10.34190/iccws.17.1.20>
- Slomka, F., & Glaß, M. (2023). *Grundlagen der Rechnerarchitektur: Von der Schaltung zum Prozessor*. Springer Fachmedien Wiesbaden. <https://doi.org/10.1007/978-3-658-36659-9>
- Su, C., & Zeng, Q. (2021). Survey of CPU cache-based side-channel attacks: Systematic analysis, security models, and countermeasures (P. Nicopolitidis, Hrsg.). *Security and Communication Networks*, 2021, 1–15. <https://doi.org/10.1155/2021/5559552>
- Van Schaik, S., Milburn, A., Osterlund, S., Frigo, P., Maisuradze, G., Razavi, K., Bos, H., & Giuffrida, C. (2019). RIDL: Rogue in-flight data load. *2019 IEEE Symposium on Security and Privacy (SP)*, 88–105. <https://doi.org/10.1109/SP.2019.00087>
- Vila, P., Kopf, B., & Morales, J. F. (2019). Theory and practice of finding eviction sets. *2019 IEEE Symposium on Security and Privacy (SP)*, 39–54. <https://doi.org/10.1109/SP.2019.00042>

- Von Faber, E. (2021). *IT und IT-Sicherheit in Begriffen und Zusammenhängen: Thematisch sortiertes Lexikon mit alphabetischem Register zum Nachschlagen*. Springer Fachmedien Wiesbaden. <https://doi.org/10.1007/978-3-658-33431-4>
- Wang, H., Tang, M., Xu, K., & Wang, Q. (2023, 3. Juni). BandwidthBreach: Unleashing covert and side channels through cache bandwidth exploitation. <https://doi.org/10.48550/arXiv.2306.01996>
- Wang, Z., & Lee, R. (2006). Covert and side channels due to processor architecture. *2006 22nd Annual Computer Security Applications Conference (ACSAC'06)*, 473–482. <https://doi.org/10.1109/ACSAC.2006.20>
- Wu, Z., Xu, Z., & Wang, H. (2012). Whispers in the hyper-space: High-speed covert channel attacks in the cloud, 159–173. Zugriff 7. Mai 2026 unter <https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/wu>
- Xiong, W., & Szefer, J. (2022). Survey of transient execution attacks and their mitigations. *ACM Computing Surveys*, 54(3), 1–36. <https://doi.org/10.1145/3442479>
- Xu, Y., Bailey, M., Jahanian, F., Joshi, K., Hiltunen, M., & Schlichting, R. (2011). An exploration of L2 cache covert channels in virtualized environments. *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, 29–40. <https://doi.org/10.1145/2046660.2046670>
- Yan, M., Choi, J., Skarlatos, D., Morrison, A., Fletcher, C., & Torrellas, J. (2018). InvisiSpec: Making speculative execution invisible in the cache hierarchy. *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 428–441. <https://doi.org/10.1109/MICRO.2018.00042>
- Yarom, Y., & Falkner, K. (2014). FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. *23rd USENIX Security Symposium (USENIX Security 14)*, 719–732. <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom>
- Zhang, Y., Juels, A., Reiter, M. K., & Ristenpart, T. (2012). Cross-VM side channels and their use to extract private keys. *Proceedings of the 2012 ACM conference on Computer and communications security*, 305–316. <https://doi.org/10.1145/2382196.2382230>

Anhang

A Mikroinstruktionen und Speicherzugriffe

Mikroinstruktionen sind eine minimale Reihe an in Hardware in der CPU implementierten Befehlen zum Ausführen von Operationen. Die Komplexität und Menge dieser Instruktionen werden bewusst möglichst gering gehalten, um das Design der Hardware zu vereinfachen. Sie beschreiben die konkreten Hardwareschritte, um komplexere Befehle der ISA auszuführen. Sie haben außerdem keinen Speicherzugriff, sondern implementierten lediglich die Interaktion mit der Hardware der Speicherkomponenten. Auch wenn diese Abstraktion zu einem entsprechenden Performanceverlust aufgrund der Übersetzung von ISA-Instruktionen führt, sind moderne CPUs mikrocodebasiert, da dies das Updaten von komplexeren Instruktionen bei Bugs oder Performanceproblemen in der ISA ermöglicht. Die Anwendung entsprechender Patches für das Verhindern von Cache-basierten Covert-Channels und Angriffen mit spekulativer Ausführung ist nur begrenzt möglich, da es sich bei diesen Angriffsvektoren um logische und konzeptionelle Eigenschaften handelt, die ausgenutzt werden. (SARANGI, 2021, S. 380, 386, 395) Instruktionen werden in derselben Reihenfolge in Mikroinstruktionen übersetzt, wie sie in die Pipeline geladen werden. Eine Branch-Prediction, um Instruktionen zu prefetchen, ist also nicht nötig. Dennoch existiert Logik, um das Ergebnis von Branch Instruktionen in Mikrocode vorherzusagen. Mosier et al., 2025, S. 6, 7 zufolge kann dieser Predictor auch für TEAs ausgenutzt werden und ist für Ursache für verschiedene bekannte Arten an TEAs.

B Grundlegende Cache Architekturen

Die zwei grundlegendsten Cache-Mapping-Methoden sind der Fully-Associative und der Direct-Mapped Cache. Bei dem Fully Associative Cache besteht der physische Adress-Speicher aus einer Reihe an Speicherzellen, in denen die Adresse des aktuell gespeicherten Blocks zusammen mit dem entsprechenden Block abgelegt wird. Hierbei sei angemerkt, dass die Blockadresse einfach als die ersten N Bits der physischen Adresse realisiert werden kann. Dadurch, dass eine Adresse in jedem Eintrag gespeichert werden kann, kann durch eine Cache-Policy immer dafür gesorgt werden, dass die meistgenutzten Blöcke im Cache bleiben. Das bedeutet, wenn eine Adresse in den Cache geladen wird und der Cache ist bereits voll, wird anhand der Cache-Policy entschieden, welcher Eintrag ersetzt wird. Dafür muss allerdings bei einem Cache-Zugriff über sehr viele Einträge iteriert werden, was zu einem deutlichen Zeitaufwand führt.

Bei einem Direct-Mapped-Cache dagegen hat jeder Block einen eindeutigen Cache-Eintrag, in dem er gespeichert werden kann. Dafür können zum Beispiel die letzten N Bit einer Adresse verwendet werden, wobei N die Länge der binären Blockadresse ist. Der Vorteil ist hier, dass beim Laden eines Blocks sofort klar ist, welcher Eintrag ersetzt werden muss. Der Nachteil ist, dass sehr häufig viel genutzte Adressen um denselben Cache-Eintrag konkurrieren, was große Verzögerungen bei Speicherzugriffen zur Folge hat. (SARANGI, 2021, S. 518–521, Handy, 1993, S. 16–18)

Aus diesem Grund wird eine Kombination der beiden Mapping-Methoden, ein SAC, verwendet. Dabei wird jede Adresse mit einer Menge an Einträgen assoziiert. Die Adresse dieses Sets hat eine Länge von $N = B - a$, wobei B die Länge der Blockadresse und a die Assoziativität des Cache ist, welche die Größe der Sets bestimmt (Größe der Sets $s = 2^a$). Der Vorteil ist, dass konkurrierende und miteinander assoziierte Adressen, zumindest bis zu einem gewissen Punkt, gleichzeitig in dem Cache bleiben können. Dafür muss allerdings eine Cache-Policy für jedes Set und damit auch gewisse Zugriffsinformationen implementiert werden. Durch die limitierte Größe des Cache-Sets muss allerdings dafür nicht durch zu viele Einträge iteriert werden. (SARANGI, 2021, S. 522–524, Handy, 1993, S. 18, 19)

C Mehrschichtiger Cache

Mehrere Cache-Layer sind nötig, da mit zunehmender Cache-Größe die Zugriffszeit steigt. Zusätzlich werden private Caches für Kerne implementiert, um die Datenraten bei Cachezugriffen nicht zwischen den Kernen aufzuteilen. (Slomka und Glaß, 2023, S. 388, SARANGI, 2021, S. 593)

Durch die mehrschichtige Architektur muss außerdem sichergestellt werden, dass Schreiboperationen auch in den darauffolgenden Schichten persistiert werden. Dazu gibt es das „write-through“ Prinzip, bei dem jede Schreiboperation direkt eine Schreiboperation in den folgenden Schichte auslöst, und das „write-back“ Prinzip, bei dem eine Schreiboperation den Block mit einem Bit als modifiziert markiert und bei dem Entfernen des Blocks aus dem Cache eine Schreiboperation aufruft. (SARANGI, 2021, S. 526)

Bei inclusive Cache-Layern muss zudem sichergestellt werden, dass eine Eviction auf allen Cache-Layern stattfindet. Bei dem Aufruf einer `clflush`-Instruktion kann die Ausführung früher fortgeführt werden, wenn sich die Adresse nicht im LLC befindet. Es reicht, den LLC anzusteuern und bei einem Cache Miss die Ausführung fortzusetzen. Bei einem Hit muss der Block aus dem Cache geflusht und ein Flush-Befehl an alle weiteren Cache-Layer gesendet werden. Daher folgt ein zeitlicher Unterschied bei der `clflush`-Instruktion, wenn die Adresse gecacht ist. (Gruss et al., 2016, S. 4)

D Spekulative Ausführung von Branch Instruktionen

Wie bereits erwähnt wird für pipelined Prozessoren ein Prefetching-Mechanismus implementiert, um die Abhängigkeit der Auslastung der Pipeline von der Speichergeschwindigkeit zu verringern. Bei Branch-Instruktionen kann sich allerdings die Folge-Instruktion ändern und damit der Control-Flow verändert werden. Das Problem ist, dass die Entscheidung, ob der Branch genommen wird und welchen Wert demnach der PC annimmt, erst in der Pipeline entschieden wird und zusätzlich von Werten im Speicher abhängig sein kann. (Kaeli und Yew, 2005, S. 14, 15, 40)

Um dennoch die Pipeline auszulasten, werden verschiedene Mechanismen verwendet, um Folgeinstruktionen auszuführen. Das Prefetching ist primär davon abhängig, ob ein Branch genommen wird oder nicht. Dabei ist zu beachten, dass sich das Prefetching

zeitlich vor der Pipeline befindet, bei einer Branch-Instruktion allerdings die direkt darauf folgende Instruktion abhängig von der Entscheidung ist. Die Vorhersage und damit auch die Änderung der nächsten Instruktionen, die vom Prefetcher geladen werden, kann an verschiedenen Stellen, mit verschiedenen Vor- und Nachteilen, dieser Architektur stattfinden. Je früher die Vorhersage stattfindet, desto weniger Informationen sind zu der Instruktion bekannt, und folglich verschlechtert sich die Genauigkeit der Vorhersage. Je später allerdings die Entscheidung getroffen wird, desto mehr Instruktionen wurden bereits transient in der Pipeline ausgeführt und müssen verworfen werden. Neben der Betrachtung, ob der Branch genommen wird, muss auch das Ziel bestimmt werden, um die korrekten Instruktionen zu laden. Je nachdem, ob die Zieladresse ein Literal oder ein dynamischer Wert ist, kann dies eine Abhängigkeit sein, von deren Auflösung abhängt, wie weit die Pipeline fortgeschritten ist. Mit der Entscheidung, ob der Branch genommen wird und was das Ziel ist, wird nicht nur das Ziel des Prefetchers geändert, sondern auch die entsprechenden Instruktionen bis zur Auflösung des Branches werden ausgeführt. Ob ein Branch genommen wird oder nicht, kann statisch z.B. anhand der konkreten Instruktion, oder dynamisch z.B. anhand vergangener Branch-Entscheidungen entschieden werden. Da bei einer Branch-Entscheidung nur einer der zwei Pfade weiter ausgeführt wird, wird diese Art von spekulativer Ausführung auch „unipath“ Execution genannt. (Kaeli und Yew, 2005, S. 40, 42–48, 136)

Dem gegenüber steht die „multipath“-Execution. Dabei werden beide Pfade ausgeführt und wenn die Branch-Entscheidung getroffen wird, werden die falsch ausgeführten Instruktionen verworfen. In einer CPU-Architektur können außerdem auch beide Konzepte implementiert werden. Dabei kann ein Predictor verwendet werden, der auch eine Konfidenz seiner Vorhersage angibt, und bei geringer Konfidenz können beide Pfade ausgeführt werden. Der Vorteil von Multipath-Execution ist, dass die Pipeline voll ausgelastet werden kann und die Ergebnisse nicht vollständig verworfen werden. Der Nachteil gegenüber unipath Execution ist wiederum, dass 2 Pfade ausgeführt werden und dies eine zusätzliche CPU-Last darstellt. Besonders bei mehreren nicht aufgelösten Branch-Entscheidungen sammeln sich diese Pfade. (Kaeli und Yew, 2005, S. 136, 137)

E Weitere technische Möglichkeiten für Covert-Channel in der Prozessorarchitektur

Viele Covert-Channel Übertragungen basieren darauf, dass die Nutzung von geteilten Ressourcen von dem Sender so ausgenutzt wird, dass diese Nutzung für den Empfänger messbar wird. Ein Beispiel für die Nutzung von geteilten Ressourcen ist die Cache-Bandbreite, das heißt die Menge an load-Instruktionen, die von der Speicher-Architektur in einer gewissen Zeit beantwortet wird. H. Wang et al., 2023 nutzen beispielsweise den Line Fill Buffer (LFB), der Speicherzugriffe speichert, die den L1 Cache verfehlen, aus, um die Bandbreite von Cache-Zugriffen zu begrenzen. Dabei werden vom Sender konkret um eine logische 1 zu übertragen Instruktionen wie „vmovntdq“ ausgeführt, die lange im LFB bleiben und damit die Ausführung von Speicherzugriffen des Empfängers in derselben Pipeline verzögern. Dafür müssen die Prozesse von Sender und Empfänger zur

gleichen Zeit auf dem gleichen physischen Kern ausgeführt werden, was ein Feature wie Simultaneous Multi-Threading (SMT) voraussetzt. Außerdem kann dieser Covert-Channel nicht im Kontext von Transient Execution genutzt werden, da das Schreiben und Lesen dafür sequenziell ausgeführt werden müssen. Ein ähnlicher Covert-Channel ohne die Verwendung von Cache-Ressourcen wird in Z. Wang und Lee, 2006, S. 2 vorgestellt. In diesem Falle werden Informationen in Form der Belegung bestimmter geteilter Recheneinheiten in demselben physischen Kern enkodiert. Durch das Ausführen vieler Multiplikations-Instruktionen kann die Pipeline eines Empfängers in demselben physischen Kern verzögert werden, wenn dieser auch Multiplikations-Instruktionen ausführt. Ein weiteres Beispiel wird in Paccagnella et al., 2021, S. 9, 10 demonstriert. Dort wird der zentrale Bus der CPU für die Kommunikation mit dem LLC für contention verwendet, wodurch dieser Covert-Channel auch für cross-core Kommunikation verwendet werden kann.

Da alle dieser Covert-Channel contentionbasiert sind, sind sie nur bedingt für den Kontext von Transient Execution anwendbar.

F Erläuterung zu verschiedenen CbCC Techniken

Zum Zeitpunkt der Veröffentlichung von „Spectre Attacks“ existierte bereits eine Menge Literatur zu cachebasierten Side-Channels, die praktisch genauso auch als CbCC verwendet werden konnten. Noch heute wird in vielen TEAs in der Literatur Side-Channel Techniken wie Flush+Reload (Yarom und Falkner, 2014) verwendet (Schwarz et al., 2019, S. 7, Van Schaik et al., 2019, S. 91, Kim, Chuang et al., 2025, S. 6, Kim, Genkin und Yarom, 2025, S. 6, Hertogh et al., o. D., S. 3).

Andere CbCC, die höhere Bandbreiten durch geringere Zugriffszeiten der unteren Cache-Layer ausnutzen, sind meistens an gewisse Bedingungen geknüpft, wie die Ausführung beider Kontexte auf demselben Kern. Ein Beispiel dafür sind Demote+Reload und Demote+Demote, welche die `cldemote`-Instruktion („CLDEMOTE — Cache Line Demote“, 2023) nutzen, um Cache-Lines aus dem L1 Cache zu evicten und in den L2 bis LLC Cache zu verschieben. Diese Instruktion wurde allerdings erst in neueren Intel-CPU-Generationen zur ISA hinzugefügt, und ist damit nicht in jeder x86-CPU verfügbar („List of x86 instructions“, 2026). Der große Vorteil von `cldemote` ist, dass keine Speicherzugriffe und keine Operationen über den LLC hinaus nötig sind. Zusätzlich implementieren neuere CPUs non-inclusive LLCs, was bisherige cross-core CbCC komplizierter macht, da Interaktionen des Zielkontexts, auf einem anderen CPU-Kern und in dessen private Caches, keine Veränderung in dem LLC zur Folge haben müssen. (Rauscher et al., 2025, S. 3, 4)

Auch für CPUs mit inclusive LLC und ohne `cldemote` wurde schon deutlich früher ein ähnlicher evictionbasierter CbCC demonstriert. Bei Maurice et al., 2015, S. 6, 7 lädt der Angreifer eine Adresse in den LLC, die aufgrund von starker Cacheauslastung evicted wird. Diese Cacheauslastung wird von dem Zielkontext ausgelöst, indem viele Adressen geschrieben und damit in allen Cache-Layern gesetzt werden. Werden keine Werte geschrieben, dann wird die Adresse des Angreifers nicht evicted, was für den Angreifer

messbar ist. Dieser Covert-Channel wurde allerdings nicht in dem Kontext von Transient Execution evaluiert und ist in diesem Kontext wahrscheinlich aufgrund der Anzahl an Schreibinstruktionen technisch nicht umsetzbar.

G Erläuterung zu der Messung, ob eine Adresse gecacht ist

Eine Grenze, mit der bestimmt wird, ob eine Adresse gecacht ist oder nicht, kann praktisch nur statistisch ermittelt werden. Diese Grenze hat erheblichen Einfluss auf die Genauigkeit des CbCC, da sie die Metrik definiert, ob ein korrekt encodierter Wert auch korrekt ausgelesen wird. Vergangene Messungen zu Cache-basierten Side-Channels (Yarom und Falkner, 2014, S. 6, 7, Rauscher et al., 2025, S. 5–7) zeigen, dass der zeitliche Unterschied zwischen den zwei betrachteten Zuständen klar unterscheidbar ist. Allerdings kann je nach Software, Hardwareumgebung und Systemauslastung die Dauer solcher Zugriffe deutlich variieren (Yarom und Falkner, 2014, S. 7). Aus diesem Grund wurde diese Grenze auch häufig manuell bestimmt. Um CbCC allerdings für verschiedene Konfigurationen nutzbar zu machen, wird in dem Test-Programm dynamisch vor der Übertragung eine Grenze ermittelt. Diese Strategie zur Ermittlung der Grenze wird in dieser Arbeit konzeptionell nicht tiefer evaluiert, sondern nur beschrieben.

Die Grenze wird auf Basis von 100 Messungen von Adressen, die vorher geladen wurden, und 100 Messungen, die vorher geflusht wurden, berechnet. Diese Adressen werden als Array allokiert. Um die Unabhängigkeit dieser Adressen sicherzustellen und ein Prefetching zu verhindern, wird jede Messung mit Index i semizufällig mit dem randomisierten Index r durchgeführt, genauso wie in Kocher et al., 2019, S. 19:

$$r = (i * 167) + 67 \mod 100$$

Dabei sei zu beachten, dass 167 und 67 Primzahlen sind und somit eine eindeutige Indizierung sicherstellen. Darüber hinaus wird angenommen, dass die Zugriffszeiten normalverteilt sind, obwohl Speicherzugriffe aus physischen Gründen eine untere Grenze haben und, zumindest in virtualisierten Umgebungen, mit geringer Wahrscheinlichkeit Zugriffe auf den Speicher sehr hohe Zugriffszeiten haben können (Yarom und Falkner, 2014, S. 7). Diese Annahme wird dennoch getroffen, um die Komplexität zu verringern.

Aus den 100 Messwerten wird jeweils das arithmetische Mittel für die Laufzeit, der geflushten Adressen f , sowie der geladenen Adressen c , gebildet und daraus folgendermaßen eine Grenze g berechnet:

$$g = c + \left(\frac{f - c}{2} \right)$$

H Abbildungen zur theoretischen Laufzeit einer Übertragung bei TEAs

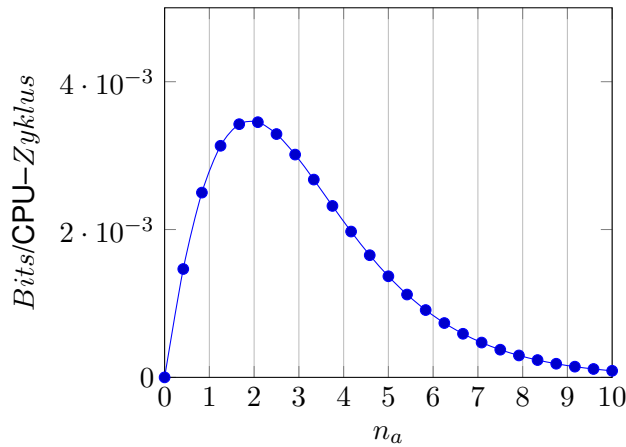


Abbildung 4: $D(n_a)$ mit $N_T = 50$, $T_T = 20$, $L_E = 100$, $T_P = 50$, $T_R = 120$ und $n_{bmax} = 8$

$$D'(n_a) = - \frac{2n_{bmax} (n_{bmax} ((\ln(2) (T_R + 2T_P) n_a - T_R) \cdot 2^{n_a} - 2T_P (2^{n_a} - 1)) - 2(N_T T_T + L_E))}{(n_{bmax} (T_R \cdot 2^{n_a} + 2T_P (2^{n_a} - 1)) + 2(N_T T_T + L_E))^2}$$

Abbildung 5: Die erste Ableitung der Durchsatz-Funktion $D(n_a)$

I Programmausschnitte des Test-Programms

```

1 inline __always_inline bool leak_data(void (*asm_encode)(uint64_t data, uint8_t *
  channel,
2                               uint8_t bits_count, uint64_t stride),
3                               uint32_t iteration, uint32_t *train_data,
4                               uint32_t train_data_len, uint64_t secret, uint8_t channel[],
5                               uint8_t bits_count, uint64_t stride)
6 {
7     if (train_data[iteration] < train_data_len)
8     {
9         asm_encode(secret, channel, bits_count, stride);
10        return true;
11    }
12    return false;
13 }

```

Listing 1: Funktion, die Transient Execution in dem Test-Programm auslöst

```

1 static inline __always_inline void asm_encode_load_bitwise(
2     uint64_t data, uint8_t *channel,
3     uint8_t bits_count, uint64_t stride)
4 {
5     __asm__ volatile("xor %%rax, %%rax\n"
6
7                     "movb $0x0, %%bl\n"
8                     "movq $0x1, %%rcx\n"
9                     "movq %[stride], %%rdx\n"
10                    "movb %[bitscount], %%sil\n"
11                    "enc0:\n"
12

```

```

13         "addb $0x1, %%b1\n"
14         "addq %%rdx, %[channel]\n"
15         "movq %[data], %%rax\n"
16         "andq %%rcx, %%rax\n"
17         "testq %%rax, %%rax\n"
18         "jz  nenc0\n"
19         "prefetcht0 ([channel])\n"
20
21         "nenc0:\n"
22         "sal $0x1, %%rcx\n"
23         "cmpb %%sil, %%b1\n"
24         "jne  enc0\n"
25         :
26         : [data] "r"(data), [channel] "r"(channel), [stride] "r"(
           stride), [bitscount] "r"(bits_count)
27         : "rax", "rbx", "rcx", "rdx", "rsi", "cc");
28     }

```

Listing 2: Funktion für die bitweise Encodierung eines Secrets in den Covert-Channel

J Cache Spezifikationen der verwendeten CPUs

Listing 3: Ausgabe von `lscpu` und `lscpu --caches` des Intel® Core™ i7-6700K

```

1  Architecture:                x86_64
2  CPU op-mode(s):              32-bit, 64-bit
3  Address sizes:                39 bits physical, 48 bits virtual
4  Byte Order:                  Little Endian
5  CPU(s):                      8
6  On-line CPU(s) list:         0-7
7  Vendor ID:                   GenuineIntel
8  Model name:                  Intel(R) Core(TM) i7-6700K CPU @ 4.00GHz
9  CPU family:                  6
10 Model:                      94
11 Thread(s) per core:          2
12 Core(s) per socket:          4
13 Socket(s):                   1
14 Stepping:                    3
15 Microcode version:           0xf0
16 CPU(s) scaling MHz:          58%
17 CPU max MHz:                 4200.0000
18 CPU min MHz:                 800.0000
19 BogomIPS:                    7999.96
20 Flags:                        fpu vme de pse tsc msr pae mce cx8 apic
                               sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe
                               syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good
                               nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl
                               vmx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe
                               popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch
                               cpuid_fault pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept vpid ept_ad
                               fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid mpx rdseed adx smap
                               clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm ida arat pln pts hwp
                               hwp_notify hwp_act_window hwp_epp vnmi md_clear flush_l1d arch_capabilities
21 Virtualization:              VT-x
22 L1d cache:                   128 KiB (4 instances)
23 L1i cache:                   128 KiB (4 instances)
24 L2 cache:                    1 MiB (4 instances)
25 L3 cache:                    8 MiB (1 instance)
26 NUMA node(s):                1

```

27	NUMA node0 CPU(s):	0-7
28	Vulnerability Gather data sampling:	Vulnerable: No microcode
29	Vulnerability Ghostwrite:	Not affected
30	Vulnerability Indirect target selection:	Not affected
31	Vulnerability Itlb multihit:	KVM: Mitigation: VMX disabled
32	Vulnerability L1tf: cache flushes, SMT vulnerable	Mitigation; PTE Inversion; VMX conditional
33	Vulnerability Mds: vulnerable	Mitigation; Clear CPU buffers; SMT
34	Vulnerability Meltdown:	Mitigation; PTI
35	Vulnerability Mmio stale data: vulnerable	Mitigation; Clear CPU buffers; SMT
36	Vulnerability Old microcode:	Not affected
37	Vulnerability Reg file data sampling:	Not affected
38	Vulnerability Retbleed:	Mitigation; IBRS
39	Vulnerability Spec rstack overflow:	Not affected
40	Vulnerability Spec store bypass: disabled via prctl	Mitigation; Speculative Store Bypass
41	Vulnerability Spectre v1: __user pointer sanitization	Mitigation; usercopy/swapgs barriers and
42	Vulnerability Spectre v2: conditional; RSB filling; PBR SB-eIBRS Not affected; BHI Not affected	Mitigation; IBRS; IBPB conditional; STIBP
43	Vulnerability Srbds:	Mitigation; Microcode
44	Vulnerability Tsa:	Not affected
45	Vulnerability Tsx async abort:	Mitigation; TSX disabled
46	Vulnerability Vmscape:	Mitigation; IBPB before exit to userspace
47		
48	NAME ONE-SIZE ALL-SIZE WAYS TYPE	LEVEL SETS PHY-LINE COHERENCY-SIZE
49	L1d 32K 128K 8 Data	1 64 1 64
50	L1i 32K 128K 8 Instruction	1 64 1 64
51	L2 256K 1M 4 Unified	2 1024 1 64
52	L3 8M 8M 16 Unified	3 8192 1 64

Listing 4: Ausgabe von `lscpu` und `lscpu -caches` des Intel® Core™ i7-7700HQ

1	Architecture:	x86_64
2	CPU op-mode(s):	32-bit, 64-bit
3	Address sizes:	39 bits physical, 48 bits virtual
4	Byte Order:	Little Endian
5	CPU(s):	8
6	On-line CPU(s) list:	0-7
7	Vendor ID:	GenuineIntel
8	Model name:	Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz
9	CPU family:	6
10	Model:	158
11	Thread(s) per core:	2
12	Core(s) per socket:	4
13	Socket(s):	1
14	Stepping:	9
15	Microcode version:	0xf8
16	CPU(s) scaling MHz:	91%
17	CPU max MHz:	3800,0000
18	CPU min MHz:	800,0000
19	BogoMIPS:	5599,85
20	Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfperf pni pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust sgx bmi1 avx2 smep bmi2 erms invpcid mpx rdseed adx

```

smmap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm ida arat pln pts
hwp hwp_notify hwp_act_window hwp_epp vnmi md_clear flush_lld
arch_capabilities
21 Virtualization: VT-x
22 L1d cache: 128 KiB (4 instances)
23 L1i cache: 128 KiB (4 instances)
24 L2 cache: 1 MiB (4 instances)
25 L3 cache: 6 MiB (1 instance)
26 NUMA node(s): 1
27 NUMA node0 CPU(s): 0-7
28 Vulnerability Gather data sampling: Mitigation; Microcode
29 Vulnerability Ghostwrite: Not affected
30 Vulnerability Indirect target selection: Not affected
31 Vulnerability Itlb multihit: KVM: Mitigation: Split huge pages
32 Vulnerability L1tf: Mitigation; PTE Inversion; VMX conditional
    cache flushes, SMT vulnerable
33 Vulnerability Mds: Mitigation; Clear CPU buffers; SMT
    vulnerable
34 Vulnerability Meltdown: Mitigation; PTI
35 Vulnerability Mmio stale data: Mitigation; Clear CPU buffers; SMT
    vulnerable
36 Vulnerability Old microcode: Not affected
37 Vulnerability Reg file data sampling: Not affected
38 Vulnerability Retbleed: Mitigation; IBRS
39 Vulnerability Spec rstack overflow: Not affected
40 Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass
    disabled via prctl
41 Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and
    __user pointer sanitization
42 Vulnerability Spectre v2: Mitigation; IBRS; IBPB conditional; STIBP
    conditional; RSB filling; PBRSE-eIBRS Not affected; BHI Not affected
43 Vulnerability Srbds: Mitigation; Microcode
44 Vulnerability Tsa: Not affected
45 Vulnerability Tsx async abort: Not affected
46 Vulnerability Vmscape: Mitigation; IBPB before exit to userspace
47
48 NAME ONE-SIZE ALL-SIZE WAYS TYPE LEVEL SETS PHY-LINE COHERENCY-SIZE
49 L1d 32K 128K 8 Data 1 64 1 64
50 L1i 32K 128K 8 Instruction 1 64 1 64
51 L2 256K 1M 4 Unified 2 1024 1 64
52 L3 6M 6M 12 Unified 3 8192 1 64

```

Listing 5: Ausagbe von lscpu und lscpu -caches des AMD A10-7850K APU

```

1 Architecture: x86_64
2 CPU op-mode(s): 32-bit, 64-bit
3 Address sizes: 48 bits physical, 48 bits virtual
4 Byte Order: Little Endian
5 CPU(s): 4
6 On-line CPU(s) list: 0-3
7 Vendor ID: AuthenticAMD
8 Model name: AMD A10-7850K APU with Radeon(TM) R7
    Graphics
9 CPU family: 21
10 Model: 48
11 Thread(s) per core: 2
12 Core(s) per socket: 2
13 Socket(s): 1
14 Stepping: 1
15 Frequency boost: enabled
16 CPU(s) scaling MHz: 46%
17 CPU max MHz: 3700.0000

```

```

18 CPU min MHz: 1700.0000
19 BogoMIPS: 7384.20
20 Flags: fpu vme de pse tsc msr pae mce cx8 apic
      sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext
      fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl nonstop_tsc cpuid
      extd_apicid aperfmperf pni pclmulqdq monitor ssse3 fma cx16 sse4_1 sse4_2
      popcnt aes xsave avx f16c lahfm_lahf_lm cmp_legacy svm extapic cr8_legacy abm sse4a
      misalignsse 3dnowprefetch osvw ibs xop skinit wdt lwp fma4 tce nodeid_msr tbm
      topoext perfctr_core perfctr_nb bpeext ptsc cpb hw_pstate ssbd vmmcall fsgsbase
      bmi1 xsaveopt arat npt lbrv svm_lock nrip_save tsc_scale vmcb_clean flushbyasid
      decodeassists pausefilter pfthreshold overflow_recov
21 Virtualization: AMD-V
22 L1d cache: 64 KiB (4 instances)
23 L1i cache: 192 KiB (2 instances)
24 L2 cache: 4 MiB (2 instances)
25 NUMA node(s): 1
26 NUMA node0 CPU(s): 0-3
27 Vulnerability Gather data sampling: Not affected
28 Vulnerability Indirect target selection: Not affected
29 Vulnerability Itlb multihit: Not affected
30 Vulnerability L1tf: Not affected
31 Vulnerability Mds: Not affected
32 Vulnerability Meltdown: Not affected
33 Vulnerability Mmio stale data: Not affected
34 Vulnerability Reg file data sampling: Not affected
35 Vulnerability Retbleed: Mitigation; untrained return thunk; SMT
      vulnerable
36 Vulnerability Spec rstack overflow: Not affected
37 Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass
      disabled via prctl
38 Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and
      __user pointer sanitization
39 Vulnerability Spectre v2: Mitigation; Retpolines; STIBP disabled;
      RSB filling; PBRSE-eIBRS Not affected; BHI Not affected
40 Vulnerability Srbds: Not affected
41 Vulnerability Tsa: Not affected
42 Vulnerability Tsx async abort: Not affected
43 Vulnerability Vmscape: Not affected
44
45 NAME ONE-SIZE ALL-SIZE WAYS TYPE LEVEL SETS PHY-LINE COHERENCY-SIZE
46 L1d 16K 64K 4 Data 1 64 1 64
47 L1i 96K 192K 3 Instruction 1 512 1 64
48 L2 2M 4M 16 Unified 2 2048 1 64

```

K Messungen des Test-Programms

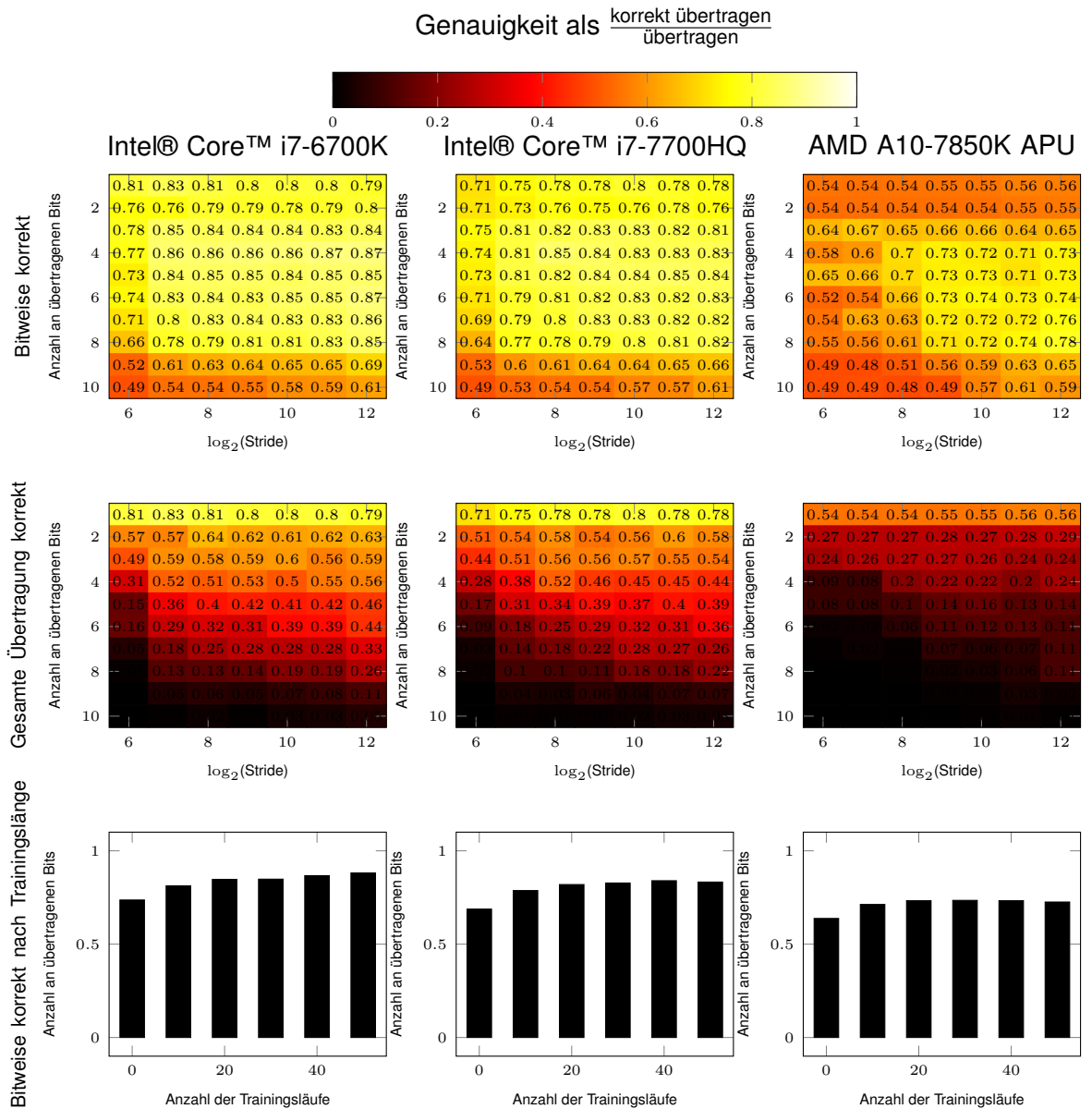


Abbildung 6: Messergebnisse des Test-Programms auf verschiedenen CPUs (Encodierung bitweise)

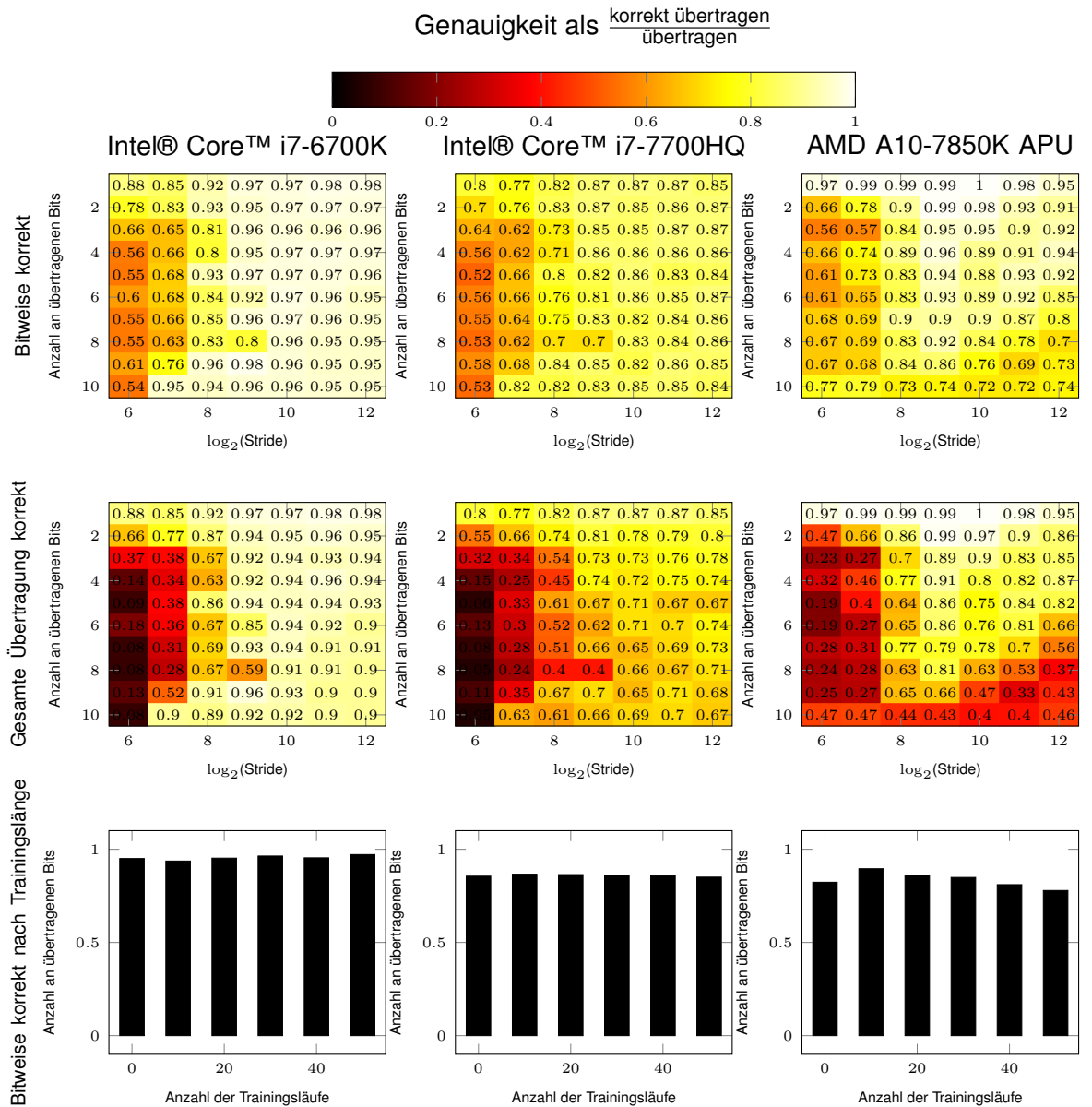


Abbildung 7: Messergebnisse des Test-Programms auf verschiedenen CPUs (Encodierung Array-Index)

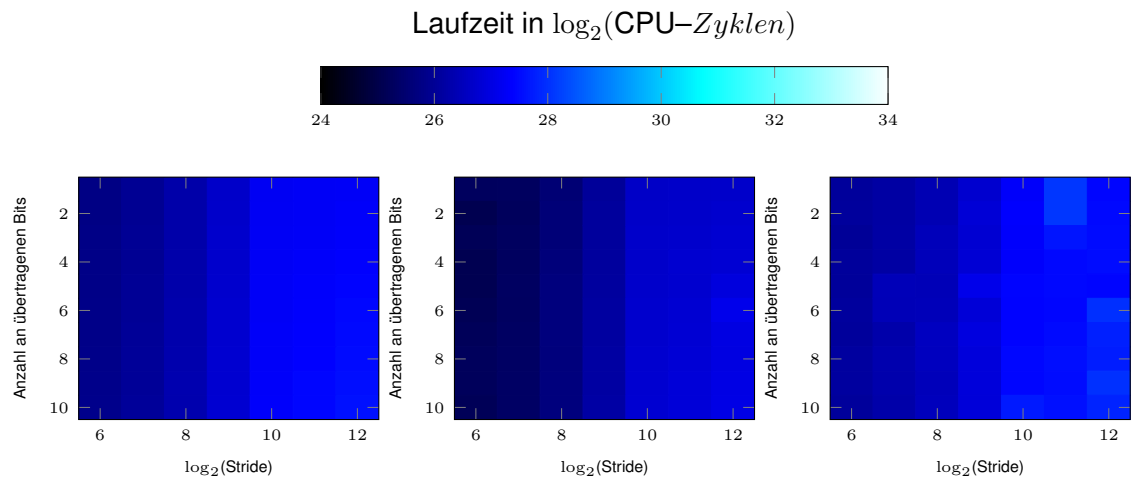


Abbildung 8: Messergebnisse zur Laufzeit des Test-Programms auf verschiedenen CPUs (Encodierung bitweise)

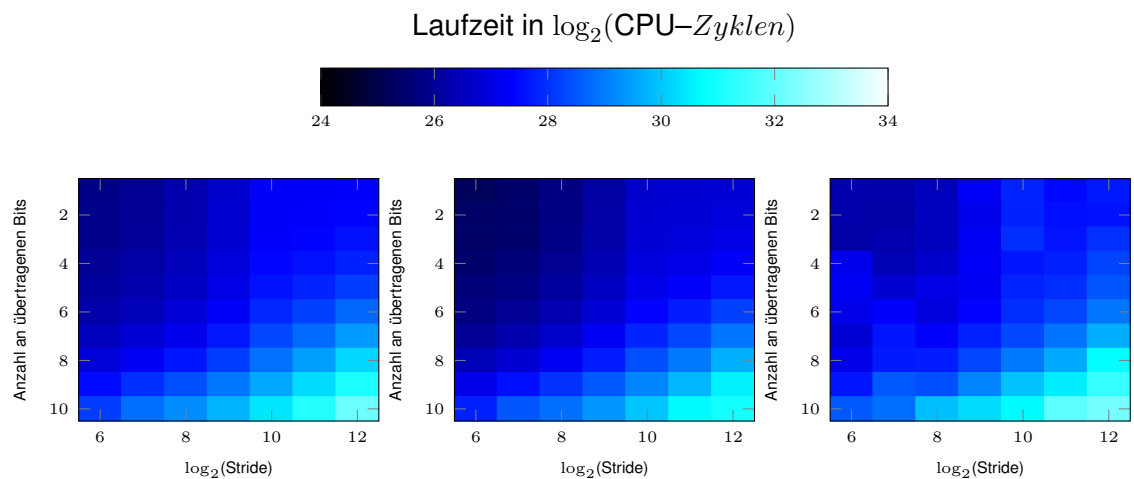


Abbildung 9: Messergebnisse zur Laufzeit des Test-Programms auf verschiedenen CPUs (Encodierung Array-Index)

Ehrenwörtliche Erklärung

Hiermit versichere ich, dass die vorliegende Arbeit von mir selbstständig und ohne unerlaubte Hilfe angefertigt worden ist, insbesondere, dass alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen sind, durch Zitate als solche kenntlich gemacht wurden. Diese Versicherung bezieht sich auch auf die in der Arbeit verwendete bildliche Darstellungen, Tabellen, Zeichnungen, Skizzen, grafischen Darstellungen und dergleichen sowie auch für die Verwendung von text- oder Code-generierenden KI-Werkzeugen als Quelle.

Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen und ist nicht veröffentlicht. Sie wurde nicht, auch nicht auszugsweise, für eine andere Prüfungs- oder Studienleistung verwendet.

Ich bin damit einverstanden, dass die Arbeit einer elektronischen Plagiatsprüfung unterzogen werden kann. Die Regelungen der Prüfungsordnung zur Täuschung habe ich zur Kenntnis genommen.

Die von mir abgegebenen schriftlichen (gebundenen) Versionen der vorliegenden Arbeit stimmen inhaltlich mit der eingereichten elektronischen Fassung überein.

Ort, 1.6.2026,